

CHAPTER 1 · WHY RUST FEELS DIFFERENT

Why Rust Feels Different

Production Rust is valuable because ownership, failure handling, concurrency, and deployment rules become explicit contracts that teams can review before release.

Opening scenario

A payments platform is moving latency-sensitive services to Rust after repeated incidents in resource ownership, failure recovery, and concurrent state updates. The business requirement is direct: make ownership, mutation authority, and fallibility explicit enough for release review before the system grows.

What Rust is asking you to state up front

- Name the owner of the data or resource.
- Say whether the next step only borrows, mutates exclusively, or takes ownership outright.
- Make fallibility and concurrency visible where the boundary actually changes.

A reading checklist

1. Who owns the data or resource after this line?
2. Does the next step need read-only access, exclusive mutation, or full ownership?
3. Is fallibility visible in the signature or hidden in control flow?
4. If concurrency appears, is the boundary message passing, shared immutable state, or synchronized shared mutation?

Mental model

Rust is not “C++ with a stricter compiler,” “C# without a GC,” or “Go with harder syntax.” The operational model is different: values have owners, borrows describe temporary access, and APIs must state whether they transfer responsibility, share read access, or require exclusive mutation.

- **Ownership** says who is responsible for cleanup.
- **Borrowing** says who may access a value temporarily.
- **Traits** describe behavior; generics usually compile via monomorphization rather than dynamic dispatch.
- **Thread boundaries** carry trait requirements like `Send` and `Sync`.

Core concepts

Rust’s core promise: control without unsafety by default

Rust wants C-like control over layout, allocation, and destruction while keeping memory unsafety out of ordinary code. Safe Rust rules out use-after-free, double-free, and data races in the safe subset. When low-level work truly needs escape hatches, you use `unsafe` in small, auditable regions and document the invariants there.

Zero-cost abstractions in practice

“Zero-cost” does not mean “free magic.” It means abstractions do not inherently add overhead relative to a hand-written equivalent. Iterator pipelines, enums, and generics often compile to code close to an explicit loop. Traits become dynamic only when you opt into trait objects; otherwise, generic code is usually monomorphized.

The senior-developer Rust learning curve

The learning curve is mostly architectural, not syntactic. Rust pushes design choices earlier: where is ownership transferred, which references may outlive which scopes, which types are allowed across threads, and where should cloning be explicit? Later chapters go deeper on borrowing, traits, and async, but this chapter's goal is to make the early discomfort interpretable.

Rust compared with C++, C#, and Go

The most useful comparison is not syntax. It is where each language places trust: C++ trusts discipline around aliasing, C# trusts a managed runtime, Go trusts a runtime plus simplified concurrency primitives, and Rust spends more compile time to reduce what must be trusted in production.

Comparison callout: what changes by background**C++ background**

RAII and move semantics carry over, but Rust turns aliasing and mutation rules into ordinary API design instead of team discipline. The biggest shift is not syntax. It is that the compiler now participates in resource-protocol review.

C# background

Rust trades runtime-managed convenience for deterministic destruction and explicit fallibility. The design question moves earlier: who owns this resource, and what does the caller have to prove before using it?

Go background

Rust keeps the lightweight service instinct but removes much of the ambient trust around shared state and cross-thread ownership. Message passing, queues, and worker handoff still exist; the ownership contract just becomes explicit.

Why this feels slower before it feels faster**The friction is concentrated at boundaries**

Small expressions are rarely the expensive part of learning Rust. APIs, shared state, queues, and worker handoff are where the language insists that ownership, mutation, and lifetime become explicit.

Your old instincts still help, but they need a new ordering

C++ instincts help with layout and RAII, C# instincts help with modeling, and Go instincts help with service decomposition. Rust's extra question is always: who owns this value right now, and who owns it after the next call?

Compiler feedback is early architectural feedback

A rejected ownership design is often cheaper than a late production incident. Rust surfaces protocol mistakes while the code is still small enough to reshape confidently.

Compile-time guarantees vs runtime checks**USUALLY COMPILE TIME**

- Move-after-use and many lifetime mistakes
- Overlapping mutable borrows
- Thread-boundary safety via trait bounds
- Missing exhaustiveness in pattern matching

STILL RUNTIME WORK

- Allocation, syscalls, and network failures
- Bounds checks when data-dependent
- Parsing invalid input and I/O availability
- Contention, cache misses, and real performance costs

Reading compiler errors productively

1. Read the violated rule first, not just the red text.
2. Find the origin span: where was the move, borrow, or trait bound introduced?
3. Change ownership shape before changing syntax.
4. Only clone or heap-share after deciding that duplication or sharing is actually correct.

A practical reading loop

1. Name the violated rule first: move, borrow exclusivity, trait bound, lifetime relationship, or exhaustiveness.
2. Find the first ownership mistake, not only the last line the compiler highlighted.
3. Rewrite the ownership story in plain English before editing the code.
4. Then choose the repair deliberately: borrow, return ownership, shorten scope, clone on purpose, or change the boundary.

error[E0382]**borrow of moved value**

Ownership already moved, so this binding is no longer the one allowed to use the value.

Typical repair: Decide whether the boundary should borrow, return ownership, or clone deliberately. The repair is about API shape before it is about syntax.

error[E0499]**cannot borrow as mutable more than once**

Rust is protecting exclusive mutation. Two active mutable paths to the same state would violate Rust's exclusive-mutation rule.

Typical repair: Shorten the first borrow, split the state, or centralize mutation in one owner instead of working around the borrow checker.

error[E0277]**type cannot be sent between threads safely**

The thread boundary demands stronger guarantees than the captured type can currently provide.

Typical repair: Move owned data, share immutable data via shared references, or introduce a thread-safe ownership model such as Arc at the boundary that actually needs it.

Production patterns

Turn parse-time bytes into owned domain values before queue, task, or thread boundaries where the caller can no longer guarantee lifetime.

Use borrowing for narrow read paths, not as a way to carry request-local state deeper into the system than it belongs.

Prefer enums, Result, and explicit ownership transitions over sentinel values, ambient mutation, or class-shaped state machines.

Start concrete, then generalize only after the second real implementation or boundary appears.

Pitfalls and tradeoffs

Cloning to quiet the borrow checker before deciding whether the callee needed ownership at all.

Recreating inheritance or shared-mutable object graphs when a struct, enum, or one-owner workflow would have fit the problem better.

Wrapping broad state in `Arc<Mutex<T>>` as the first move instead of deciding who should own mutation.

Treating 'zero-cost abstraction' like 'zero work.' Iteration, hashing, parsing, allocation, and contention still cost what they cost.

⚠ WARNING

A borrow-checker error is not an obstacle to work around. It is exposing an ownership protocol you have not stated clearly enough yet. Treat that as design feedback, especially in production code.

Examples

Example 1-1. `zero_cost_pipeline.rs` — *an iterator pipeline with no extra abstraction tax*

```
fn critical_count(readings: &[i32], threshold: i32) -> usize {
    readings
        .iter()
        .copied()
        .filter(|reading| *reading >= threshold)
        .count()
}

fn main() {
    let readings = vec![42, 87, 91, 63, 99];
    let count = critical_count(&readings, 80);
    println!("critical count = {}", count);
}
```

OUTPUT

```
critical count = 3
```

 TRY IT

Confirm the baseline output, then change the threshold or the input data and note how the pipeline stays readable while the result stays correct. The CPU still walks the slice, evaluates the predicate, branches, and counts matches — zero-cost abstractions remove abstraction tax, not the cost of real work.

Example 1-2. `ownership_thread_handoff.rs` — *ownership handoff into a worker thread*

```
use std::thread;

fn main() {
    let job = String::from("rebuild-search-index");

    let handle = thread::spawn(move || {
        println!("worker started: {}", job);
    });

    handle.join().unwrap();
}
```

OUTPUT

```
worker started: rebuild-search-index
```

 NOTE

The `move` closure consumes captured non-`Copy` data so the worker owns what it needs, which makes the handoff explicit instead of leaving lifetime questions to convention. Queues, threads, and async tasks are all ownership boundaries: borrow locally when you can, but hand off owned work items when execution may outlive the current scope.

Summary

- Rust feels different because it makes resource protocols part of the type-checked API instead of part of team folklore.
- The goal is not abstract safety rhetoric. It is earlier feedback on ownership, mutation, and thread-boundary mistakes.
- Zero-cost abstractions remove abstraction tax, not the real cost of parsing, allocation, synchronization, or IO.
- Once you can read compiler errors as design notes, later chapters get much easier to use well.

CHAPTER 1 · EXERCISES

Chapter 01 Exercises

Exercises to turn “Rust feels strict” into a repeatable review habit around ownership, diagnostics, and production design.

How to use this page

These exercises are intentionally solution-free in the repository. Treat the objective, starter prompt, acceptance criteria, and hints as a working spec. Aim for explicit ownership choices, concrete tradeoff language, and short architectural explanations another senior engineer could review quickly.

Suggested working loop

1. Restate the exercise in ownership language before you write code.
2. Decide whether the boundary wants borrowing, ownership transfer, or explicit cloning.
3. Work the lab, then explain the result by naming the ownership or borrowing rule involved.
4. Write down one tradeoff you accepted: allocation, duplication, indirection, or API complexity.

EXERCISE 1 WARM-UP COMPREHENSION**Translate a familiar lifecycle pattern into Rust****OBJECTIVE**

Map one resource-management idea from C++, C#, or Go to idiomatic Rust ownership and cleanup semantics.

STARTER PROMPT

Choose one familiar lifecycle pattern and restate it as a Rust ownership design for a resource owner named `Session` or `LogWriter`.

- Start from a single owner that is responsible for cleanup.
- Decide which methods should take `&self`, which should take `&mut self`, and which should consume `self`.
- Explain where deterministic cleanup happens and why it does not depend on a garbage collector or inheritance.

ACCEPTANCE CRITERIA

- Your design identifies a single owner responsible for cleanup.
- You explain why borrowed methods do or do not allow mutation.
- Cleanup is deterministic and does not rely on a garbage collector or inheritance.

HINTS

- Think in terms of a struct plus methods, not a class hierarchy.
- If cleanup must always happen, describe the role of `Drop` even if you do not fully implement it yet.

EXERCISE 2 CODE READING**Find the zero-cost part and the real runtime cost****OBJECTIVE**

Identify which parts of a Rust abstraction are compile-time structure and which parts still do real work at runtime.

STARTER PROMPT

Study an iterator-based function that filters high-priority jobs from a slice, then annotate which parts are compile-time abstraction structure and which parts still cost CPU cycles.

- Call out which parts are likely monomorphized or optimized as abstraction structure.
- List the runtime work that still exists: iteration, branching, cache behavior, and any downstream materialization cost.
- Explain where dynamic dispatch would appear if the return type changed to a trait object.

ACCEPTANCE CRITERIA

- You identify iterator adapters and `impl Iterator` as abstraction mechanisms that do not imply dynamic dispatch by default.
- You name the runtime work that still exists: iteration, branching, cache behavior, and any downstream allocation if materialized.
- You explain that a trait object would opt into dynamic dispatch.

HINTS

- Separate 'how the code is expressed' from 'what the machine still has to do.'
- Look for where values are actually stored, allocated, or compared.

EXERCISE 3 IMPLEMENTATION**Make fallibility explicit instead of implicit****OBJECTIVE**

Replace a sentinel-value style API with Rust's explicit `Option` and `Result` modeling.

STARTER PROMPT

Implement

```
parse_limit(input: Option<&str>) -> Result<usize, &'static str>
so missing input uses the default value 100, "0" is invalid, non-
numeric input is invalid, and valid positive integers return
Ok(limit).
```

- Do not use `panic!`, `unwrap`, or sentinel values such as `-1`.
- Keep the failure mode explicit in the type system so callers cannot ignore it accidentally.

ACCEPTANCE CRITERIA

- The function returns `Result<usize, &'static str>` exactly.
- Missing input becomes the default value without error.
- Invalid input is represented as an `Err`, not as a magic number or log-only failure.

HINTS

- Handle `Option` first, then parse the string branch.
- The type system should make invalid states harder to ignore at call sites.

EXERCISE 4 DEBUGGING OR REFACTORING**Interpret three compiler diagnostics and propose repairs****OBJECTIVE**

Practice reading Rust diagnostics as ownership and concurrency design feedback.

STARTER PROMPT

For each diagnostic, explain the rule Rust is enforcing, sketch the likely shape of the buggy code, and propose one or two valid repair strategies.

- E0382: borrow of moved value: request
- E0499: cannot borrow buffer as mutable more than once at a time
- E0277: `Rc<String>` cannot be sent between threads safely

ACCEPTANCE CRITERIA

- Your explanation of E0382 mentions ownership transfer and alternatives such as borrowing, returning ownership, or intentional cloning.
- Your explanation of E0499 mentions exclusive mutable access and proposes scope shortening or data-structure redesign.
- Your explanation of E0277 mentions thread-safety trait bounds and distinguishes `Rc` from thread-safe sharing approaches such as `Arc`.

HINTS

- Focus on the violated rule first, then the syntax.
- A good repair changes ownership shape before reaching for shared mutable state.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO**Design a worker pipeline with explicit ownership boundaries****OBJECTIVE**

Choose Rust-native boundaries for parsing, validation, dispatch, and cross-thread communication in a production-style service.

STARTER PROMPT

You are designing an ingestion service with the flow

```
socket bytes -> parse -> validate -> route to workers -> emit metrics.
```

- At what boundary do bytes become owned domain values?
- What data is borrowed only locally?
- What types are allowed to cross thread boundaries?
- Where do you use `Result` or enums to represent failure and state?

ACCEPTANCE CRITERIA

- You place ownership boundaries before cross-thread or queued work.
- You avoid defaulting to a single global `Arc<Mutex<HashMap<...>>>` as the first design move.
- You make error handling explicit with `Result`, enums, or both.
- You justify at least one tradeoff involving performance, safety, or maintainability.

HINTS

- Prefer message passing of owned work items over broad shared mutability.
- If you need sharing, explain why the sharing is semantically real rather than just convenient.

Runnable lab · Exercise 3

Implement the explicit-fallibility version of `parse_limit`. The checker expects a default value for missing input, an explicit rejection for zero, and a successful positive parse. Keep the type as `Result<usize, &'static str>`, handle `None` first, then

validate and parse the `Some` branch.

Example 1-3. `parse_limit_lab.rs` — starter — make it produce the target output

```
fn parse_limit(input: Option<&str>) -> Result<usize, &'static str> {
    match input {
        None => Ok(0),
        Some(raw) => match raw.parse::<usize>() {
            Ok(limit) => Ok(limit),
            Err(_) => Err("invalid number"),
        },
    }
}

fn main() {
    println!("default = {:?}" , parse_limit(None));
    println!("zero = {:?}" , parse_limit(Some("0")));
    println!("value = {:?}" , parse_limit(Some("25")));
}
```

TARGET OUTPUT (AFTER YOUR FIX)

```
default = Ok(100)
zero = Err("limit must be greater than 0")
value = Ok(25)
```

NOTE

The listing above is the starter: it returns the default as `Ok(0)` and accepts zero. Your task is to make it produce the target output instead.

Questions to ask for each error

E0382

What moved, and should this call site borrow, return ownership, or clone intentionally?

E0499

Which mutable borrow stayed alive too long, and how can the scopes or data shape be narrowed?

E0277

Which thread boundary requires `Send` or `Sync`, and is sharing actually necessary here?

Review questions

- Why does Rust often feel harder at API boundaries than inside small local functions?
- What is the practical difference between a generic function using traits and a trait object?
- Name one category of bug Rust usually catches at compile time and one category it cannot remove from runtime reality.
- Why is 'just clone it' sometimes correct and sometimes a design smell?
- How would you explain `Send` and `Sync` to a teammate coming from Go or C#?

What success looks like

By the end of these exercises, you should be able to explain Rust strictness in ownership-contract terms, diagnose common move, borrow, and thread-boundary failures, and defend at least one production design in Rust-native language instead of translating it mechanically from C++, C#, or Go.

CHAPTER 2 · THE RUST MENTAL MODEL

The Rust Mental Model

Reliable Rust services depend on a precise model of values, ownership transfer, drops, stack and heap storage, expressions, and references. This chapter defines that model for code review and debugging.

Builds on Chapter 1

Chapter 1 explained why Rust feels different at a high level. This chapter turns that intuition into an operational model: who owns a value, what actually moves, what gets dropped, and why lifetimes are proofs about references rather than a runtime memory system.

Opening scenario

A message-ingestion service parses inbound bytes, enriches records, and dispatches owned jobs to workers. The business requirement is a precise value lifecycle: where data is created, which component owns it, where heap storage is used, and when cleanup occurs.

Operational checklist

1. What value exists here, and which binding owns it right now?
2. If the value is heap-backed, which part is inline metadata and which part owns separate storage?
3. Where does scope end, and therefore where does deterministic cleanup happen?
4. If a reference appears, which owner keeps it valid?

Mental model

Values are the thing; bindings are scope-local names

A Rust `let` binding is not a little object wrapper with secret identity. It is a name in a scope. The value can move, be borrowed temporarily, or be dropped when its owner goes out of scope.

Moves transfer responsibility, not data by magic

For non-`Copy` types such as `String`, `Vec<T>`, and most structs, assignment or argument passing usually transfers ownership. After the move, the previous binding is simply no longer the owner.

Lifetimes describe borrowed reach, not object survival

A lifetime annotation does not keep data alive and does not behave like a garbage collector. It only states a compile-time relationship: this reference must not outlive the owner it points into.

Core concepts

Values, bindings, moves, and drops

Think operationally: values exist, bindings name them, ownership determines who will run cleanup, moves transfer ownership, and `Drop` runs when the owner leaves scope. This model is simpler than class identity plus hidden runtime memory management, but it is more explicit.

```
let b = a;
```

For a non-Copy value, ownership moved and the original binding no longer owns cleanup.

Stack vs heap allocation

Do not confuse the owner with the storage behind it. A `String` value is a small stack-resident handle, but its bytes live on the heap. A `Vec<T>` owns a heap buffer; the `Vec` metadata itself is a fixed-size value. Heap allocation buys flexibility, not free performance.

```
let payload = String::from("ok");
```

The handle is local data; the string bytes are heap-backed.

Immutability by default

Bindings are immutable unless you write `mut`. This makes state changes stand out in APIs and local code. It does not mean Rust is globally immutable; it means mutation is opt-in and therefore easier to audit.

```
let mut y = 1;
```

Mutation is explicit at the binding where it is needed.

Expressions, statements, and blocks

Most constructs in Rust produce values. `if`, `match`, and blocks can return a result. This makes it natural to build values with local scratch state and then expose only the final immutable result.

```
let timeout_ms = if bursty { 200 } else { 50 };
```

Control flow can compute a value directly instead of widening mutation.

RAII and deterministic destruction

Rust inherits the RAII spirit familiar to C++ engineers, but makes the ownership rules visible in ordinary code. Scope exit triggers deterministic destruction in reverse lexical order. That matters for files, sockets, locks, buffers, and transaction guards.

```
{ let file = open_log(); }
```

Cleanup happens when the owner leaves scope, not when a GC eventually notices it.

Lifetimes as compile-time reasoning, not garbage collection

Lifetimes are attached to references, not owned values. They let the compiler verify that borrowed data is still valid where used. They are not runtime tags, object regions, or reachability roots.

```
fn pick<'a>(left: &'a str, right: &'a str) -> &'a str
```

The annotation constrains the returned reference; it does not extend any owner's lifetime.

Comparison callout: translating prior instincts**C++ background**

RAII and move semantics will feel familiar, but Rust refuses casual aliasing patterns that C++ often permits. References are more constrained, and safe code cannot quietly rely on discipline alone.

C# background

There is no GC extending object reachability behind the scenes. If a value must survive, some owner must hold it. If a reference is returned, the compiler proves the referent outlives that use.

Go background

Go hides many storage decisions behind escape analysis and a garbage collector. Rust exposes ownership more directly: borrowed local views stay local, while owned values cross threads, queues, and subsystem boundaries.

Two corrections that remove a lot of confusion

A move is not “copy then invalidate” as a user model

The useful model is simpler: ownership transferred. For small fixed-size `Copy` values the data is copied. For non-`Copy` types, think of the old binding as no longer owning the value. That is the rule that matters when reading APIs and diagnostics.

A lifetime does not keep anything alive

Owned values live until their owner is dropped. Lifetimes constrain references that point at those values. If the owner is dropped, no annotation can keep a reference to it valid.

Expressions, statements, and blocks in one pass

```
let retries = 3;
```

A statement performs work in the enclosing scope. It does not become the value of that scope.

```
if bursty { 200 } else { 50 }
```

An expression produces a value. In Rust, `if`, `match`, and blocks are often value-producing tools, not only control-flow syntax.

```
{ let base = 40; base + 2 }
```

The last line without a semicolon becomes the block’s value. Add a trailing semicolon and the block evaluates to `()` instead.

**RULE OF THUMB**

When a lifetime error appears, repair ownership before you reach for annotations. Usually the real choice is one of two shapes: borrow from caller-owned input that already lives long enough, or return an owned value so the function transfers data instead of a reference. Randomly adding `'a` to a signature rarely fixes the model, because lifetimes describe valid borrowing relationships; they do not extend how long an owner lives.

Production patterns

Use borrowed views for short local work, but convert to owned values before task queues, worker handoff, cache storage, or long-lived structs.

Prefer block expressions to assemble validated immutable values. Local mutation inside a short block is often clearer than a wider mutable scope.

Treat `Drop` as the last line of defense for releasing resources, not as a general business-logic callback mechanism.

Model ownership at subsystem boundaries first; performance work is easier once the movement and lifetime of data are explicit.

Pitfalls and tradeoffs

Confusing a binding with object identity. Rebinding a name is not the same thing as preserving one owner.

Using a reference as if it could extend lifetime. References describe access; they do not keep the referenced value alive.

Marking wide state as `mut` early and then fighting borrow conflicts that were really scope-design problems.

Trying to return references to function-local data instead of deciding who should own the result.

Putting slow, fallible, or order-sensitive business logic inside `Drop` instead of keeping destructors small and unsurprising.

Adding lifetime annotations before the ownership model is clear enough to justify them.

**WARNING**

Deterministic destruction is powerful, but it is not magic. Destructors should release resources and maintain invariants, not hide slow network calls, lock acquisition chains, or critical business logic that is hard to reason about during unwinding and shutdown.

Examples

Example 2-1. `move_and_drop_timeline.rs` — *move a value and observe deterministic drop*

```
struct FileHandle {
    name: String,
}

impl Drop for FileHandle {
    fn drop(&mut self) {
        println!("drop {}", self.name);
    }
}

fn ship(handle: FileHandle) {
    println!("shipping {}", handle.name);
}

fn main() {
    let handle = FileHandle {
        name: String::from("audit.log"),
    };

    println!("before move");
    ship(handle);
    println!("after ship");
}
```

OUTPUT

```
before move
shipping audit.log
drop audit.log
after ship
```

**TRY IT**

The binding in `main` stops owning the value after the function call, and cleanup runs when the new owner leaves scope. Trace the output by hand, then rename the file or move more fields through the handoff to make the ownership timeline concrete.

Example 2-2. `expression_blocks_and_heap.rs` — build owned heap data with narrow mutation

```
fn main() {
    let queue_depth = 2048;

    let status = if queue_depth > 1024 { "hot" } else { "steady" };

    let labels = {
        let mut labels = Vec::with_capacity(8);
        labels.push(String::from("ingest"));
        labels.push(String::from("priority"));
        labels
    };

    let capacity = {
        let heap_slots = labels.capacity();
        heap_slots
    };

    println!("status = {}", status);
    println!("capacity = {}", capacity);
}
```

OUTPUT

```
status = hot
capacity = 8
```

**TRY IT**

The outer bindings stay immutable while a short inner block uses mutation to assemble the final value. Confirm the expected output, then vary the block contents to see how narrow mutation still yields an owned final value.

Summary

- Rust gets calmer when you treat bindings as names and ownership as the cleanup contract.
- Stack and heap explain storage placement; ownership explains who is responsible for the value that uses it.
- Immutability by default and expression-oriented blocks push mutation into narrower, more reviewable scopes.
- RAII in Rust is explicit and deterministic, which is exactly why `Drop` should stay small and unsurprising.
- Lifetimes talk only about borrowed references. They never extend the life of owned data.

CHAPTER 2 · EXERCISES

Chapter 02 Exercises

Exercises for reasoning about moves, drops, allocation shape, expression-oriented code, and lifetime constraints.

How to use this page

Work from ownership and scope first, then syntax. For each exercise, explain who owns the value, when it can be borrowed, and where destruction happens. The repository keeps exercises solution-free on purpose: treat the objective and acceptance criteria as your spec.

Suggested working loop

1. Name the owner first, then describe the borrow or move.
2. Separate storage shape from ownership shape before you optimize the design in your head.
3. Explain where scope ends and what gets dropped there.
4. If a reference appears, state which owner keeps it valid.

EXERCISE 1 WARM-UP COMPREHENSION

Predict the ownership timeline

OBJECTIVE

Practice narrating which binding owns a value after each move and where `Drop` is guaranteed to run.

STARTER PROMPT

Read a short ownership-transfer sequence with `service`, `alias`, and `consume`, then narrate the owner after each move in plain English.

- Which binding owns the string after each line?
- At what scope does the string get dropped?
- Which extra line could you add that would fail to compile, and why?

ACCEPTANCE CRITERIA

- You explain that ownership moves from `service` to `alias`, then from `alias` into `consume`.
- You identify the end of `consume` as the drop point for the owned `String`.
- Your failing-line example correctly refers to using a moved binding after ownership transfer.

HINTS

- For non-`Copy` types, assignment normally transfers ownership.
- Describe ownership one scope at a time instead of thinking about hidden object identity.

EXERCISE 2 CODE READING**Map stack and heap storage precisely****OBJECTIVE**

Explain which parts of a composite value are inline and which parts own heap allocations.

STARTER PROMPT

Analyze a `Batch` struct that stores a fixed array plus a `Vec<String>`, then write a short note about which pieces are inline and which pieces own heap allocations.

- Which parts of `Batch` are stored inline in the local value?
- What heap allocations exist?
- Which value owns each heap allocation?

ACCEPTANCE CRITERIA

- You identify the fixed-size array as inline in the `Batch` value.
- You explain that `Vec<String>` contains inline metadata while its element buffer is heap allocated.
- You note that each `String` element owns its own heap-backed bytes.

HINTS

- Separate the outer struct layout from the storage used by each field's owned data.
- A `Vec<T>` is a fixed-size handle that owns a separate buffer.

EXERCISE 3 IMPLEMENTATION**Refactor imperative mutation into expression-oriented Rust****OBJECTIVE**

Rewrite a function so temporary state stays narrow and the final result is produced by expressions.

STARTER PROMPT

Refactor

`classify(depth: usize) -> (&'static str, usize)` so the tuple comes directly from an expression instead of a wide mutable temporary.

- Keep the return type the same.
- Prefer `if` and block expressions.
- Do not introduce heap allocation or cloning.

ACCEPTANCE CRITERIA

- Your final version avoids the wide `mut scaled` binding.
- The result is still `(&'static str, usize)`.
- The control flow is expression-oriented rather than early-return driven everywhere.

HINTS

- An `if` expression can return a tuple.
- A short inner block can compute a value without widening mutable scope.

EXERCISE 4 DEBUGGING OR REFACTORING**Explain the lifetime bug instead of fighting the annotation****OBJECTIVE**

Diagnose why returning a reference to local data fails and propose correct repairs.

STARTER PROMPT

Diagnose a function that tries to return the first line of a local `String` by reference, then explain why that borrow cannot outlive the owner.

- What rule Rust is enforcing
- Why adding a random lifetime annotation does not solve it
- Two valid repairs with different tradeoffs

ACCEPTANCE CRITERIA

- You explain that the returned reference would outlive the local `String` owner.
- You state that lifetimes describe valid borrowing relationships and do not extend object lifetime.
- You propose at least two real repairs, such as returning an owned `String` or borrowing from caller-provided input.

HINTS

- Ask who owns `text` and when that owner is dropped.
- A repair is valid only if the referenced data outlives the returned reference.

EXERCISE 5 DESIGN OR PRODUCTION SCENARIO**Choose owned versus borrowed data at a queue boundary****OBJECTIVE** **STARTER PROMPT**

Make an ownership plan for a realistic parser-to-worker pipeline.

You are designing an ingest path with the flow

```
&[u8] request bytes -> parse headers -> build Event -> send to worker queue -> persist -> emit metrics.
```

- Which data stays borrowed only inside parsing?
- Which data becomes owned inside `Event` before the queue boundary?
- Where is cloning acceptable, and where is it a smell?
- What cleanup should rely on ordinary scope exit or `Drop`?

ACCEPTANCE CRITERIA

- You keep borrowed data local to parsing or validation where possible.
- You convert data that crosses the queue boundary into owned values.
- You justify cloning as an explicit economic choice rather than a default response.
- You describe cleanup in terms of ownership and scope, not hidden runtime behavior.

HINTS

- A worker queue is usually an ownership boundary.
- Try to name the owner at each subsystem edge.

Runnable lab · Exercise 3

Refactor the starter so `classify` returns its tuple from an expression. The checker looks for the correct branch values and for the wide mutable temporary to disappear. Aim for an `if` expression that returns `("hot", depth / 2)` or `("steady", depth + 50)` directly.

Example 2-3. `classify_lab.rs` — starter — refactor toward an expression

```
fn classify(depth: usize) -> (&'static str, usize) {
    let mut scaled = 0;

    if depth > 1000 {
        scaled = depth / 2;
        return ("hot", scaled);
    }

    scaled = depth + 50;
    ("steady", scaled)
}

fn main() {
    let (status_a, value_a) = classify(120);
    let (status_b, value_b) = classify(1200);
    println!("{}", status_a, value_a);
    println!("{}", status_b, value_b);
}
```

TARGET OUTPUT (AFTER YOUR REFACTOR)

```
steady 170
hot 600
```

Review questions

- What is the difference between a value and a binding in Rust?
- Where do `Vec<T>` metadata and `Vec<T>` elements usually live?
- What does a lifetime annotation constrain, and what does it not do?
- Why can block expressions reduce mutation pressure in production code?
- When is `Drop` useful, and what kinds of logic should usually stay out of it?

What success looks like

By the end of this page, you should be able to narrate ownership transfer without hand-waving, distinguish stack layout from heap-backed ownership precisely, refactor toward expression-oriented Rust when it improves clarity, and explain why a lifetime error is really an ownership error with references on top.

CHAPTER 3

Project Structure and Tooling

Cargo, crates, workspaces, modules, features, dependencies, and a practical workflow

Overview

Cargo, crates, workspaces, modules, features, dependencies, and a practical workflow

Examples

Example 3-1. `project_structure_module_visibility.rs` — *Project structure module visibility*

```
mod config {
    pub struct AppConfig {
        pub service_name: String,
        bind_addr: String,
    }

    impl AppConfig {
        pub fn new(service_name: &str, bind_addr: &str) -> Self {
            Self {
                service_name: service_name.to_string(),
                bind_addr: bind_addr.to_string(),
            }
        }

        pub fn bind_addr(&self) -> &str {
            &self.bind_addr
        }
    }
}

fn main() {
    let config = config::AppConfig::new("api", "127.0.0.1:8080");
    println!("{}", config.service_name, config.bind_addr());
}
```

Example 3-2. `project_structure_feature_flags.rs` — *Project structure feature flags*

```
// In a real crate, a Cargo feature would usually select this at compile time.
const METRICS_ENABLED: bool = false;

fn metrics_backend() -> &'static str {
    if METRICS_ENABLED {
        "prometheus"
    } else {
        "disabled"
    }
}

fn main() {
    println!("metrics backend = {}", metrics_backend());
}
```

Exercises

Design workspace layout, feature matrices, dependency policy, and the command pack for CI

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 4

Ownership, Borrowing, and Lifetimes

Ownership as the core design tool, borrowing rules, lifetimes, elision, and ownership-shaped APIs

Overview

Ownership as the core design tool, borrowing rules, lifetimes, elision, and ownership-shaped APIs

Examples

Example 4-1. `ownership_borrowing_shared_mutable.rs` — *Ownership borrowing shared mutable*

```
fn request_size(line: &str) -> usize {
    line.len()
}

fn append_trace(line: &mut String, trace_id: &str) {
    line.push_str(" trace=");
    line.push_str(trace_id);
}

fn main() {
    let mut request = String::from("GET /ready");
    let len_before = request_size(&request);
    append_trace(&mut request, "abc-123");

    println!("len before = {}", len_before);
    println!("request = {}", request);
}
```

Example 4-2. `ownership_borrowing_lifetimes.rs` — *Ownership borrowing lifetimes*

```
fn first_segment(path: &str) -> &str {
    path.split('/').find(|segment| !segment.is_empty()).unwrap_or("root")
}

fn pick_longer<'a>(left: &'a str, right: &'a str) -> &'a str {
    if left.len() >= right.len() { left } else { right }
}

fn make_audit_label(service: &str, key: &str) -> String {
    format!("{}", service, key)
}

fn main() {
    let route = String::from("/api/health");
    let segment = first_segment(&route);
    let longer = pick_longer("job", "request-id");
    let label = make_audit_label("gateway", segment);

    println!("segment = {}", segment);
    println!("longer = {}", longer);
    println!("label = {}", label);
}
```

Exercises

Repair borrow-checker failures, annotate lifetimes only where needed, and design borrowed-input APIs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 5

Pointers, References, and Ownership Inside Structs

Owned fields, borrowed fields, Box/Rc/Arc, owned buffers with views, and safe alternatives to self-references

Overview

Owned fields, borrowed fields, Box/Rc/Arc, owned buffers with views, and safe alternatives to self-references

Examples

Example 5-1. `ownership_structs_owned_views.rs` — *Ownership structs owned views*

```
struct LogLine {
    raw: String,
}

impl LogLine {
    fn new(raw: &str) -> Self {
        Self {
            raw: raw.to_string(),
        }
    }

    fn level(&self) -> &str {
        self.raw.split(':').next().unwrap_or("UNKNOWN")
    }

    fn message(&self) -> &str {
        self.raw
            .split_once(':')
            .map(|(_, message)| message.trim())
            .unwrap_or("")
    }
}

fn main() {
    let line = LogLine::new("INFO: user signed in");
    println!("level = {}", line.level());
    println!("message = {}", line.message());
}
```

Example 5-2. ownership_structs_pointer_choices.rs — Ownership structs pointer choices

```

use std::rc::Rc;
use std::sync::Arc;
use std::thread;

struct ParserConfig {
    delimiter: Box<str>,
}

#[derive(Clone)]
struct UiTemplate {
    name: Rc<str>,
}

#[derive(Clone)]
struct SharedSchema {
    version: Arc<str>,
}

fn main() {
    let parser = ParserConfig {
        delimiter: "=>".into(),
    };
    let template = UiTemplate {
        name: Rc::from("invoice"),
    };
    let template_copy = template.clone();
    let schema = SharedSchema {
        version: Arc::from("v2"),
    };
    let schema_for_worker = schema.clone();

    let handle = thread::spawn(move || {
        format!("thread schema = {}", schema_for_worker.version)
    });

    println!("box delimiter = {}", parser.delimiter);
    println!("rc clones = {}", Rc::strong_count(&template_copy.name));
    println!("{}", handle.join().unwrap());
}

```

Exercises

Choose field ownership, avoid lifetime-heavy structs, expose safe views, and replace self-references with stable layouts

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 6

Pointers, References, and Ownership Inside Vectors

Vec<T> ownership, reference invalidation, reallocation hazards, index handles, stable-address patterns, slices, and safe mutation while iterating

Overview

Vec<T> ownership, reference invalidation, reallocation hazards, index handles, stable-address patterns, slices, and safe mutation while iterating

Examples

Example 6-1. ownership_vectors_indices.rs — Ownership vectors indices

```
#[derive(Debug)]
struct Task {
    name: String,
    ready: bool,
}

fn push_task(tasks: &mut Vec<Task>, name: &str) -> usize {
    let index = tasks.len();
    tasks.push(Task {
        name: name.to_string(),
        ready: true,
    });
    index
}

fn mark_not_ready(tasks: &mut [Task], index: usize) {
    tasks[index].ready = false;
}

fn main() {
    let mut tasks = Vec::with_capacity(1);

    let billing = push_task(&mut tasks, "billing");
    push_task(&mut tasks, "search");
    push_task(&mut tasks, "indexer");
    mark_not_ready(&mut tasks, billing);

    println!("task = {}", tasks[billing].name);
    println!("ready = {}", tasks[billing].ready);
    println!("total = {}", tasks.len());
}
```


Example 6-2. `ownership_vectors_boxed_stable.rs` — *Ownership vectors boxed stable*

```

use std::ptr;

#[derive(Debug)]
struct Job {
    name: String,
    attempts: usize,
    needs_retry: bool,
}

fn main() {
    let mut jobs: Vec<Box<Job>> = Vec::new();

    jobs.push(Box::new(Job {
        name: String::from("billing"),
        attempts: 1,
        needs_retry: true,
    }));

    let first_ptr: *const Job = &*jobs[0];

    jobs.push(Box::new(Job {
        name: String::from("search"),
        attempts: 1,
        needs_retry: false,
    }));
    jobs.push(Box::new(Job {
        name: String::from("indexer"),
        attempts: 1,
        needs_retry: false,
    }));

    let same_address = ptr::eq(first_ptr, &*jobs[0]);

    let retry_indices: Vec<usize> = jobs
        .iter()
        .enumerate()
        .filter_map(|(index, job)| job.needs_retry.then_some(index))
        .collect();

    for index in retry_indices {
        jobs[index].attempts += 1;
        jobs[index].needs_retry = false;
    }

    println!("same address = {}", same_address);
    println!("attempts = {}", jobs[0].attempts);
    println!("retry pending = {}", jobs.iter().filter(|job| job.needs_retry).count());
}

```

Exercises

Repair reallocation hazards, switch from references to indices, and mutate vectors safely under production constraints

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 7

Copying Data vs Cloning Data

Move semantics, Copy vs Clone, clone-on-write, Rc/Arc cloning costs, allocation discipline, and receiver choices

Overview

Move semantics, Copy vs Clone, clone-on-write, Rc/Arc cloning costs, allocation discipline, and receiver choices

Examples

Example 7-1. `copying_data_moves_copy_clone.rs` — *Copying data moves copy clone*

```
#[derive(Debug, Copy, Clone)]
struct RequestId(u64);

#[derive(Debug, Clone)]
struct ServiceConfig {
    name: String,
    retries: usize,
}

impl ServiceConfig {
    fn name(&self) -> &str {
        &self.name
    }

    fn bump_retries(&mut self) {
        self.retries += 1;
    }

    fn replace_name(self, next: &str) -> Self {
        Self {
            name: next.to_string(),
            retries: self.retries,
        }
    }
}

fn main() {
    let request = RequestId(42);
    let request_copy = request;

    let mut live = ServiceConfig {
        name: String::from("ingest-v1"),
        retries: 1,
    };
    live.bump_retries();

    let replaced = live.clone().replace_name("ingest-v2");

    println!("request id copy = {}", request_copy.0);
    println!("live retries = {}", live.retries);
    println!("builder consumed = {}", replaced.name());
    println!("current name = {}", live.name());
}
```

Example 7-2. `copying_data_cow_arc.rs` — *Copying data cow arc*

```
use std::borrow::Cow;
use std::sync::Arc;

fn normalize_label(input: &str) -> Cow<'_, str> {
    if input.bytes().all(|b| matches!(b, b'a'..=b'z' | b'0'..=b'9' | b'-'')) {
        Cow::Borrowed(input)
    } else {
        Cow::Owned(input.trim().to_ascii_lowercase().replace(' ', "-"))
    }
}

fn main() {
    let borrowed = normalize_label("ready");
    let owned = normalize_label("Mixed Case");
    let schema = Arc::new(String::from("event.v1"));
    let worker_schema = Arc::clone(&schema);

    println!("borrowed = {}", borrowed);
    println!("owned = {}", owned);
    println!("strong count = {}", Arc::strong_count(&worker_schema));
}
```

Exercises

Classify move/copy/clone operations, implement Clone manually, remove unnecessary clones, and choose Copy safely

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 8

Undefined Behavior and Unsafe Rust

What Rust considers UB, the role of unsafe, raw pointers, aliasing, data races, uninitialized memory, FFI, and safe abstractions over unsafe internals

Overview

What Rust considers UB, the role of unsafe, raw pointers, aliasing, data races, uninitialized memory, FFI, and safe abstractions over unsafe internals

Examples

Example 8-1. `unsafe_rust_fill_window.rs` — *Unsafe rust fill window*

```
fn fill_window(buf: &mut [u8], start: usize, len: usize, value: u8) {
    assert!(start <= buf.len());
    assert!(start + len <= buf.len());

    let ptr = buf.as_mut_ptr();

    for offset in 0..len {
        unsafe {
            // SAFETY:
            // - \
```

Example 8-2. `unsafe_rust_maybe_uninit.rs` — *Unsafe rust maybe uninit*

```
use std::mem::MaybeUninit;

fn build_header(tag: u8, size: u8) -> [u8; 4] {
    let mut bytes = MaybeUninit::<[u8; 4]>::uninit();
    let ptr = bytes.as_mut_ptr() as *mut u8;

    unsafe {
        // SAFETY:
        // - each element is written exactly once before \
```

Exercises

Audit unsafe abstractions, identify UB risks in raw-pointer code, repair MaybeUninit misuse, and design safe wrappers around unsafe boundaries

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 9

Smart Pointers and Pinning

Box, Rc, Arc, Cell, RefCell, Mutex, Weak, Pin, and the ownership tradeoffs behind them

Overview

Smart pointers encode ownership count, mutation authority, thread sharing, and address stability. This chapter uses Box, Rc, Arc, Weak, interior mutability, and Pin as production design choices.

Examples

Example 9-1. smart_pointers_tree.rs — Smart pointers tree

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let root = Rc::new(Node {
        name: String::from("root"),
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });

    let leaf = Rc::new(Node {
        name: String::from("leaf"),
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });

    root.children.borrow_mut().push(Rc::clone(&leaf));
    *leaf.parent.borrow_mut() = Rc::downgrade(&root);

    let parent_name = leaf
        .parent
        .borrow()
        .upgrade()
        .map(|node| node.name.clone())
        .unwrap_or_else(|| String::from("none"));

    println!("root children = {}", root.children.borrow().len());
    println!("leaf parent = {}", parent_name);
    println!("root strong = {}", Rc::strong_count(&root));
}
```

Example 9-2. smart_pointers_pin_poll.rs — *Smart pointers pin poll*

```

use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

struct Countdown {
    remaining: u8,
}

impl Future for Countdown {
    type Output = &'static str;

    fn poll(self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        let this = self.get_mut();

        if this.remaining == 0 {
            Poll::Ready("done")
        } else {
            this.remaining -= 1;
            Poll::Pending
        }
    }
}

struct NoopWake;

impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn main() {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut task = Box::pin(Countdown { remaining: 2 });

    loop {
        match Future::poll(task.as_mut(), &mut cx) {
            Poll::Ready(value) => {
                println!("ready = {}", value);
                break;
            }
            Poll::Pending => {
                println!("pending = {}", task.as_ref().get_ref().remaining);
            }
        }
    }
}

```

Exercises

Choose pointer types deliberately, break cycles with Weak, and explain why pinned futures exist

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 10

Arrays, Slices, and Vectors

Fixed-size arrays, slice-first APIs, `Vec<T>` internals, capacity planning, mutation patterns, and contiguous-storage tradeoffs

Overview

Contiguous storage choices affect allocation, cache locality, and API flexibility. This chapter defines arrays for fixed shape, slices for borrowed access, and vectors for owned growth.

Examples

Example 10-1. `arrays_slices_vectors_slice_api.rs` — *Arrays slices vectors slice api*

```
fn tail_sum(values: &[u64], take: usize) -> u64 {
    let start = values.len().saturating_sub(take);
    values[start..].iter().copied().sum()
}

fn main() {
    let fixed = [3_u64, 5, 8, 13];
    let dynamic = vec![1_u64, 2, 3, 4, 5, 6];

    println!("fixed tail = {}", tail_sum(&fixed, 2));
    println!("dynamic tail = {}", tail_sum(&dynamic, 3));
}
```

Example 10-2. `arrays_slices_vectors_capacity.rs` — *Arrays slices vectors capacity*

```
fn collect_even_scaled(ids: &[u32]) -> Vec<u32> {
    let mut out = Vec::with_capacity(ids.len());

    for &id in ids {
        if id % 2 == 0 {
            out.push(id * 10);
        }
    }

    out
}

fn main() {
    let ids = [10_u32, 11, 12, 13, 14];
    let mut out = collect_even_scaled(&ids);

    println!("len = {}", out.len());
    println!("can fit two more = {}", out.len() + 2 <= out.capacity());

    out.extend([200, 220]);
    println!("last = {}", out.last().copied().unwrap());
}
```

Exercises

Practice slice-first APIs, preallocation decisions, and choosing array, slice, `Vec`, or inline-first storage under production constraints

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 11

Hash Maps and Sets

HashMap and HashSet fundamentals, borrowed lookups, Entry API, BTreeMap tradeoffs, stable ordering, and hasher choices

Overview

Hash maps and sets sit on critical lookup paths. This chapter covers key ownership, borrowed lookup, entry-based updates, deterministic output, and hashing policy for production data access.

Examples

Example 11-1. `hash_maps_sets_entry_api.rs` — *Hash maps sets entry api*

```
use std::collections::HashMap;

fn record_hit(counts: &mut HashMap<String, usize>, route: String) {
    let slot = counts.entry(route).or_insert(0);
    *slot += 1;
}

fn main() {
    let mut counts = HashMap::with_capacity(4);

    record_hit(&mut counts, String::from("api"));
    record_hit(&mut counts, String::from("api"));
    record_hit(&mut counts, String::from("billing"));

    println!("api = {}", counts.get("api").copied().unwrap_or(0));
    println!("billing = {}", counts.get("billing").copied().unwrap_or(0));
    println!("routes = {}", counts.len());
}
```

Example 11-2. `hash_maps_sets_borrowed_lookup_ordered.rs` — *Hash maps sets borrowed lookup ordered*

```
use std::collections::{BTreeMap, HashMap, HashSet};

fn main() {
    let mut active = HashSet::new();
    active.insert(String::from("api"));
    active.insert(String::from("worker"));

    let mut counts = HashMap::new();
    counts.insert(String::from("worker"), 3);
    counts.insert(String::from("api"), 2);
    counts.insert(String::from("billing"), 1);

    let api_count = counts.get("api").copied().unwrap_or(0);
    let has_worker = active.contains("worker");

    let ordered: BTreeMap<_, _> = counts.into_iter().collect();
    let ordered_text = ordered
        .iter()
        .map(|(name, count)| format!("{}", name, count))
        .collect::<Vec<_>>()
        .join(",");

    println!("api count = {}", api_count);
    println!("has worker = {}", has_worker);
    println!("ordered = {}", ordered_text);
}
```

Exercises

Use Entry API deliberately, implement borrowed lookups, and choose between HashMap, HashSet, and ordered tree variants

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 12

Matrices and Multidimensional Data

Dense flat storage, row-major vs column-major layout, views, const generics, sparse formats, and HPC interop boundaries

Overview

Dense flat storage, row-major vs column-major layout, views, const generics, sparse formats, and HPC interop boundaries

Examples

Example 12-1. matrices_row_major_dense.rs — Matrices row major dense

```

#[derive(Debug)]
struct Matrix<T> {
    rows: usize,
    cols: usize,
    data: Vec<T>,
}

impl<T: Clone> Matrix<T> {
    fn from_elem(rows: usize, cols: usize, value: T) -> Self {
        Self {
            rows,
            cols,
            data: vec![value; rows * cols],
        }
    }
}

impl<T> Matrix<T> {
    fn offset(&self, row: usize, col: usize) -> usize {
        row * self.cols + col
    }

    fn get(&self, row: usize, col: usize) -> Option<&T> {
        if row < self.rows && col < self.cols {
            Some(&self.data[self.offset(row, col)])
        } else {
            None
        }
    }

    fn set(&mut self, row: usize, col: usize, value: T) {
        let index = self.offset(row, col);
        self.data[index] = value;
    }

    fn row(&self, row: usize) -> Option<&[T]> {
        if row < self.rows {
            let start = row * self.cols;
            Some(&self.data[start..start + self.cols])
        } else {
            None
        }
    }
}

fn main() {
    let mut m = Matrix::from_elem(2, 3, 0_i32);
    m.set(0, 0, 10);
    m.set(0, 1, 20);
    m.set(0, 2, 30);
    m.set(1, 0, 4);
    m.set(1, 1, 5);
    m.set(1, 2, 6);

    let row1_sum: i32 = m.row(1).unwrap().iter().copied().sum();

    println!("last in row0 = {}", m.get(0, 2).copied().unwrap());
    println!("row1 sum = {}", row1_sum);
}

```

Example 12-2. `matrices_views_const_generics.rs` — *Matrices views const generics*

```

struct MatrixView<'a, T> {
    data: &'a [T],
    rows: usize,
    cols: usize,
    stride: usize,
    offset: usize,
}

impl<'a, T> MatrixView<'a, T> {
    fn get(&self, row: usize, col: usize) -> Option<&T> {
        if row < self.rows && col < self.cols {
            let index = self.offset + row * self.stride + col;
            self.data.get(index)
        } else {
            None
        }
    }
}

fn trace<const N: usize>(matrix: [[i32; N]; N]) -> i32 {
    let mut sum = 0;
    let mut i = 0;

    while i < N {
        sum += matrix[i][i];
        i += 1;
    }

    sum
}

fn main() {
    let backing = vec![
        1_i32, 2, 3, 4,
        5, 6, 7, 8,
        9, 10, 11, 12,
    ];

    let view = MatrixView {
        data: &backing,
        rows: 2,
        cols: 2,
        stride: 4,
        offset: 5,
    };

    let fixed = [[3_i32, 1], [2, 4]];

    println!("view corner = {}", view.get(1, 1).copied().unwrap());
    println!("fixed trace = {}", trace(fixed));
}

```

Exercises

Implement a row-major matrix wrapper, replace nested vectors, design no-copy views, and choose dense or sparse layouts deliberately

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 13

Arena Allocation and Region-Based Memory

Bump allocators, arena-backed ASTs, region lifetimes, generational handles, slabs, and locality tradeoffs

Overview

Arena allocation supports workloads that create many related values under one lifecycle. This chapter uses region ownership and stable handles to reduce allocation overhead and simplify teardown.

Examples

Example 13-1. arena_allocation_bump_scratch.rs — Arena allocation bump scratch

```

struct Bump<const N: usize> {
    buf: [u8; N],
    used: usize,
}

impl<const N: usize> Bump<N> {
    fn new() -> Self {
        Self {
            buf: [0; N],
            used: 0,
        }
    }

    fn alloc_bytes(&mut self, bytes: &[u8]) -> Option<(usize, usize)> {
        let end = self.used.checked_add(bytes.len())?;
        if end > N {
            return None;
        }

        let start = self.used;
        self.buf[start..end].copy_from_slice(bytes);
        self.used = end;
        Some((start, end))
    }

    fn slice(&self, range: (usize, usize)) -> &[u8] {
        &self.buf[range.0..range.1]
    }

    fn used(&self) -> usize {
        self.used
    }

    fn reset(&mut self) {
        self.used = 0;
    }
}

fn main() {
    let mut arena = Bump::<32>::new();
    let first = arena.alloc_bytes(b"arena123").unwrap();
    let _second = arena.alloc_bytes(b"logs").unwrap();

    println!("used = {}", arena.used());
    println!("first = {}", std::str::from_utf8(arena.slice(first)).unwrap());

    arena.reset();
    println!("after reset = {}", arena.used());
}

```


Example 13-2. arena_allocation_index_ast.rs — Arena allocation index ast

```

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct ExprId(usize);

#[derive(Debug)]
enum Expr {
    Number(i64),
    Add(ExprId, ExprId),
    Mul(ExprId, ExprId),
}

#[derive(Default)]
struct ExprArena {
    nodes: Vec<Expr>,
}

impl ExprArena {
    fn alloc(&mut self, expr: Expr) -> ExprId {
        let id = ExprId(self.nodes.len());
        self.nodes.push(expr);
        id
    }

    fn get(&self, id: ExprId) -> &Expr {
        &self.nodes[id.0]
    }

    fn eval(&self, id: ExprId) -> i64 {
        match self.get(id) {
            Expr::Number(value) => *value,
            Expr::Add(left, right) => self.eval(*left) + self.eval(*right),
            Expr::Mul(left, right) => self.eval(*left) * self.eval(*right),
        }
    }

    fn len(&self) -> usize {
        self.nodes.len()
    }
}

fn main() {
    let mut arena = ExprArena::default();

    let two = arena.alloc(Expr::Number(2));
    let three = arena.alloc(Expr::Number(3));
    let four = arena.alloc(Expr::Number(4));
    let sum = arena.alloc(Expr::Add(two, three));
    let root = arena.alloc(Expr::Mul(sum, four));

    println!("value = {}", arena.eval(root));
    println!("nodes = {}", arena.len());
}

```

Exercises

Build a tiny arena-backed AST, compare Rc graphs with arena handles, and reason about lifetime-grouped memory

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 14

Interfaces in Rust: Traits

Traits as Rust's interface system, trait bounds, associated types, dyn Trait, dispatch choices, and object safety

Overview

Trait interfaces define capability contracts across library and service boundaries. This chapter covers bounds, associated types, generic dispatch, trait objects, and object safety as reviewable API choices.

Examples

Example 14-1. `traits_bounds_associated_types.rs` — *Traits bounds associated types*

```
trait RetryPolicy {
    type Decision;

    fn decide(&self, attempts: u32) -> Self::Decision;

    fn label(&self) -> &'static str {
        "policy"
    }

    fn should_log(&self) -> bool {
        true
    }
}

struct FixedLimit {
    max: u32,
}

impl RetryPolicy for FixedLimit {
    type Decision = bool;

    fn decide(&self, attempts: u32) -> Self::Decision {
        attempts < self.max
    }

    fn label(&self) -> &'static str {
        "fixed-limit"
    }
}

fn evaluate<P>(policy: &P, attempts: u32) -> String
where
    P: RetryPolicy<Decision = bool>,
{
    format!("{}", => {}, policy.label(), policy.decide(attempts))
}

fn main() {
    let policy = FixedLimit { max: 3 };

    println!("{}", evaluate(&policy, 2));
    println!("log = {}", policy.should_log());
}
```

Example 14-2. traits_dyn_plugin_pipeline.rs — Traits dyn plugin pipeline

```

trait Plugin {
    fn name(&self) -> &'static str;
    fn run(&self, input: &str) -> String;
}

struct Uppercase;

struct Prefix {
    value: &'static str,
}

impl Plugin for Uppercase {
    fn name(&self) -> &'static str {
        "uppercase"
    }

    fn run(&self, input: &str) -> String {
        input.to_uppercase()
    }
}

impl Plugin for Prefix {
    fn name(&self) -> &'static str {
        "prefix"
    }

    fn run(&self, input: &str) -> String {
        format!("{}", self.value, input)
    }
}

fn run_all(plugins: &[Box<dyn Plugin>], input: &str) -> Vec<String> {
    plugins
        .iter()
        .map(|plugin| format!("{}", plugin.name(), plugin.run(input)))
        .collect()
}

fn main() {
    let plugins: Vec<Box<dyn Plugin>> = vec![
        Box::new(Uppercase),
        Box::new(Prefix { value: "svc-" }),
    ];

    for line in run_all(&plugins, "rust") {
        println!("{}", line);
    }
}

```

Exercises

Translate familiar interfaces into traits, choose static or dynamic dispatch, and repair object-safety mistakes

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 15

OOP Models in Rust

Composition over inheritance, traits and dyn Trait, enums, state patterns, visitors, and Rust-native encapsulation

Overview

Composition over inheritance, traits and dyn Trait, enums, state patterns, visitors, and Rust-native encapsulation

Examples

Example 15-1. oop_models_composition_traits.rs — *Oop models composition traits*

```

mod billing {
    pub struct Invoice {
        id: u64,
        cents: u64,
    }

    impl Invoice {
        pub fn new(id: u64, cents: u64) -> Self {
            Self { id, cents }
        }

        pub fn id(&self) -> u64 {
            self.id
        }

        pub fn cents(&self) -> u64 {
            self.cents
        }
    }
}

trait Notifier {
    fn send(&self, invoice: &billing::Invoice) -> String;
}

struct EmailNotifier {
    prefix: &'static str,
}

impl Notifier for EmailNotifier {
    fn send(&self, invoice: &billing::Invoice) -> String {
        format!("{}", self.prefix, invoice.id())
    }
}

struct BillingService<N> {
    notifier: N,
    sent: usize,
}

impl<N: Notifier> BillingService<N> {
    fn process(&mut self, invoice: &billing::Invoice) -> String {
        self.sent += 1;
        format!("invoice = {} cents = {}", invoice.id(), invoice.cents())
    }

    fn notify(&self, invoice: &billing::Invoice) -> String {
        self.notifier.send(invoice)
    }
}

fn main() {
    let mut service = BillingService {
        notifier: EmailNotifier {
            prefix: "queued email for ",
        },
        sent: 0,
    };

    let invoice = billing::Invoice::new(41, 1250);
    let notification = service.notify(&invoice);

    println!("{}", service.process(invoice));
}

```

```
println!("notification = {}", notification);
println!("sent = {}", service.sent);
}
```

Example 15-2. oop_models_enum_state_machine.rs — *Oop models enum state machine*

```
#[derive(Debug)]
enum Document {
    Draft { title: String },
    Review { title: String, reviewer: String },
    Published { title: String, slug: String },
}

impl Document {
    fn request_review(self, reviewer: &str) -> Self {
        match self {
            Document::Draft { title } => Document::Review {
                title,
                reviewer: reviewer.to_string(),
            },
            state => state,
        }
    }

    fn publish(self) -> Self {
        match self {
            Document::Review { title, .. } => {
                let slug = title.to_lowercase().replace(' ', "-");
                Document::Published { title, slug }
            }
            state => state,
        }
    }

    fn state(&self) -> &'static str {
        match self {
            Document::Draft { .. } => "draft",
            Document::Review { .. } => "review",
            Document::Published { .. } => "published",
        }
    }

    fn slug(&self) -> &str {
        match self {
            Document::Published { slug, .. } => slug,
            _ => "none",
        }
    }
}

fn main() {
    let doc = Document::Draft {
        title: String::from("Rust OOP"),
    }
    .request_review("mina")
    .publish();

    println!("state = {}", doc.state());
    println!("slug = {}", doc.slug());
}
```

Exercises

Replace inheritance with composition, choose enum or trait-based polymorphism, and refactor state machines into Rust-native designs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 16

Domain-Driven Design in Rust

Entities, value objects, aggregates, repositories, invariants, event sourcing, and DDD boundaries

Overview

Domain-driven Rust encodes business rules in types, constructors, aggregate boundaries, and repository interfaces. This chapter keeps invalid states and persistence concerns out of core domain code.

Examples

Example 16-1. ddd_order_aggregate.rs — Ddd order aggregate

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct OrderId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct CustomerId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
struct Quantity(u32);

impl Quantity {
    fn new(value: u32) -> Result<Self, DomainError> {
        if value == 0 {
            return Err(DomainError::InvalidQuantity);
        }
        Ok(Self(value))
    }

    fn get(self) -> u32 {
        self.0
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
struct MoneyCents(u64);

impl MoneyCents {
    fn new(value: u64) -> Self {
        Self(value)
    }

    fn get(self) -> u64 {
        self.0
    }
}

#[derive(Debug, Clone, PartialEq, Eq)]
struct Sku(String);

impl Sku {
    fn new(value: &str) -> Result<Self, DomainError> {
        if value.trim().is_empty() {
            return Err(DomainError::EmptySku);
        }
        Ok(Self(value.to_string()))
    }
}

#[derive(Debug)]
struct OrderLine {
    sku: Sku,
    qty: Quantity,
    unit_price: MoneyCents,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum OrderStatus {
    Draft,
    Submitted,
}

#[derive(Debug)]
enum DomainError {
    InvalidQuantity,

```

```

    EmptySku,
    EmptyOrder,
    CannotModifySubmittedOrder,
}

#[derive(Debug)]
struct Order {
    id: OrderId,
    customer_id: CustomerId,
    status: OrderStatus,
    lines: Vec<OrderLine>,
}

impl Order {
    fn new(id: OrderId, customer_id: CustomerId) -> Self {
        Self {
            id,
            customer_id,
            status: OrderStatus::Draft,
            lines: Vec::new(),
        }
    }

    fn add_line(&mut self, sku: Sku, qty: Quantity, unit_price: MoneyCents) -> Result<(), DomainError> {
        if self.status == OrderStatus::Submitted {
            return Err(DomainError::CannotModifySubmittedOrder);
        }

        self.lines.push(OrderLine {
            sku,
            qty,
            unit_price,
        });

        Ok(())
    }

    fn submit(&mut self) -> Result<(), DomainError> {
        if self.lines.is_empty() {
            return Err(DomainError::EmptyOrder);
        }

        self.status = OrderStatus::Submitted;
        Ok(())
    }

    fn total_cents(&self) -> u64 {
        self.lines
            .iter()
            .map(|line| line.qty.get() as u64 * line.unit_price.get())
            .sum()
    }

    fn status(&self) -> &'static str {
        match self.status {
            OrderStatus::Draft => "draft",
            OrderStatus::Submitted => "submitted",
        }
    }
}

fn main() {
    let mut order = Order::new(OrderId(1001), CustomerId(7));

    order

```

```
.add_line(  
    Sku::new("BOOK-1").unwrap(),  
    Quantity::new(2).unwrap(),  
    MoneyCents::new(1500),  
)  
.unwrap();  
  
order  
    .add_line(  
        Sku::new("PEN-9").unwrap(),  
        Quantity::new(3).unwrap(),  
        MoneyCents::new(400),  
    )  
    .unwrap();  
  
order.submit().unwrap();  
  
println!("lines = {}", order.lines.len());  
println!("total cents = {}", order.total_cents());  
println!("state = {}", order.status());  
}
```

Example 16-2. ddd_event_sourced_account.rs — Ddd event sourced account

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct AccountId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum AccountEvent {
    Opened { opening_balance_cents: i64 },
    Deposited { cents: i64 },
    Withdrawn { cents: i64 },
}

#[derive(Debug)]
enum DomainError {
    AlreadyOpened,
    NotOpen,
    InsufficientFunds,
}

#[derive(Debug)]
struct Account {
    id: AccountId,
    balance_cents: i64,
    version: usize,
    is_open: bool,
}

impl Account {
    fn rehydrate(id: AccountId, history: &[AccountEvent]) -> Result<Self, DomainError> {
        let mut account = Self {
            id,
            balance_cents: 0,
            version: 0,
            is_open: false,
        };

        for event in history {
            account.apply(*event)?;
            account.version += 1;
        }

        Ok(account)
    }

    fn apply(&mut self, event: AccountEvent) -> Result<(), DomainError> {
        match event {
            AccountEvent::Opened {
                opening_balance_cents,
            } => {
                if self.is_open {
                    return Err(DomainError::AlreadyOpened);
                }

                self.balance_cents = opening_balance_cents;
                self.is_open = true;
                Ok(())
            }
            AccountEvent::Deposited { cents } => {
                if !self.is_open {
                    return Err(DomainError::NotOpen);
                }

                self.balance_cents += cents;
                Ok(())
            }
        }
    }
}

```

```

        AccountEvent::Withdrawn { cents } => {
            if !self.is_open {
                return Err(DomainError::NotOpen);
            }

            if self.balance_cents < cents {
                return Err(DomainError::InsufficientFunds);
            }

            self.balance_cents -= cents;
            Ok(())
        }
    }
}

fn main() {
    let history = vec![
        AccountEvent::Opened {
            opening_balance_cents: 1000,
        },
        AccountEvent::Deposited { cents: 400 },
        AccountEvent::Withdrawn { cents: 150 },
    ];

    let account = Account::rehydrate(AccountId(7), &history).unwrap();

    println!("events = {}", history.len());
    println!("balance cents = {}", account.balance_cents);
    println!("version = {}", account.version);
}

```

Exercises

Implement newtypes, encode aggregate invariants, and design repository seams for sync and async workloads

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 17

Refactoring Toward Idiomatic Rust

Refactor C++-style, C#-style, and Go-style Rust into clearer ownership, errors, traits, enums, async seams, and testable APIs

Overview

Refactor C++-style, C#-style, and Go-style Rust into clearer ownership, errors, traits, enums, async seams, and testable APIs

Examples

Example 17-1. `refactoring_result_owned_api.rs` — *Refactoring result owned api*

```
#[derive(Debug)]
enum ConfigError {
    MissingPort,
    InvalidPort,
}

#[derive(Debug)]
struct Endpoint {
    service: String,
    bind: String,
}

impl Endpoint {
    fn requires_tls(&self) -> bool {
        self.bind.ends_with(":443")
    }
}

fn parse_port(raw: Option<&str>) -> Result<u16, ConfigError> {
    let text = raw.ok_or(ConfigError::MissingPort)?;
    text.parse::<u16>().map_err(|_| ConfigError::InvalidPort)
}

fn load_endpoint(service: &str, host: &str, port_raw: Option<&str>) -> Result<Endpoint, ConfigError> {
    let port = parse_port(port_raw)?;

    Ok(Endpoint {
        service: service.to_string(),
        bind: format!("{:}:{:}", host, port),
    })
}

fn main() {
    let endpoint = load_endpoint("billing", "127.0.0.1", Some("443")).unwrap();

    println!("endpoint = {}@{}", endpoint.service, endpoint.bind);
    println!("requires tls = {}", endpoint.requires_tls());
}
```

Example 17-2. `refactoring_traits_enums_testable.rs` — *Refactoring traits enums testable*

```

#[derive(Debug, Clone, Copy)]
enum DeliveryMode {
    Immediate,
    Retry,
}

trait Notifier {
    fn notify(&self, order_id: u64) -> String;
}

struct EmailNotifier;

impl Notifier for EmailNotifier {
    fn notify(&self, order_id: u64) -> String {
        format!("email:order-{}", order_id)
    }
}

struct OrderProcessor<N> {
    notifier: N,
    processed: usize,
}

impl<N: Notifier> OrderProcessor<N> {
    fn new(notifier: N) -> Self {
        Self {
            notifier,
            processed: 0,
        }
    }

    fn process(&mut self, order_id: u64, mode: DeliveryMode) -> String {
        self.processed += 1;

        match mode {
            DeliveryMode::Immediate => format!("charged = order-{}", order_id),
            DeliveryMode::Retry => format!("queued = order-{}", order_id),
        }
    }

    fn notify(&self, order_id: u64) -> String {
        self.notifier.notify(order_id)
    }
}

fn main() {
    let mut processor = OrderProcessor::new(EmailNotifier);
    let order_id = 7_u64;

    println!("{}", processor.process(order_id, DeliveryMode::Immediate));
    println!("notified = {}", processor.notify(order_id));
    println!("processed = {}", processor.processed);
}

```

Exercises

Refactor panic-based code into Result-based code, reduce lifetime noise, remove unnecessary clones, and improve test seams

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 18

Generics Instead of Templates

Rust generics vs templates, monomorphization, trait bounds, where clauses, associated types, const generics, and performance tradeoffs

Overview

Generics support reusable Rust APIs without sacrificing concrete contracts. This chapter covers trait bounds, associated types, const generics, and monomorphization costs for production libraries.

Examples

Example 18-1. generics_batch_bounds.rs — *Generics batch bounds*

```

trait Keyed {
    fn key(&self) -> u64;
}

#[derive(Debug)]
struct Job {
    id: u64,
    name: String,
}

impl Keyed for Job {
    fn key(&self) -> u64 {
        self.id
    }
}

#[derive(Debug)]
struct Batch<T> {
    items: Vec<T>,
}

impl<T> Batch<T> {
    fn new(items: Vec<T>) -> Self {
        Self { items }
    }

    fn len(&self) -> usize {
        self.items.len()
    }
}

fn first_key<T>(batch: &Batch<T>) -> Option<u64>
where
    T: Keyed,
{
    batch.items.first().map(|item| item.key())
}

fn main() {
    let batch = Batch::new(vec![
        Job {
            id: 10,
            name: String::from("parse"),
        },
        Job {
            id: 22,
            name: String::from("persist"),
        },
    ]);

    println!("len = {}", batch.len());
    println!("first key = {}", first_key(&batch).unwrap());
}

```

Example 18-2. `generics_associated_types_const.rs` — *Generics associated types const*

```

fn hex_digit(nibble: u8) -> u8 {
    match nibble {
        0..=9 => b'0' + nibble,
        10..=15 => b'A' + (nibble - 10),
        _ => b'?',
    }
}

trait Encoder {
    type Output;

    fn encode(&self, input: &[u8]) -> Self::Output;
}

struct HexPair;

impl Encoder for HexPair {
    type Output = [u8; 2];

    fn encode(&self, input: &[u8]) -> Self::Output {
        let byte = input[0];
        [hex_digit(byte >> 4), hex_digit(byte & 0x0F)]
    }
}

#[derive(Debug)]
struct FixedWindow<T, const N: usize> {
    items: [T; N],
}

impl<const N: usize> FixedWindow<u32, N> {
    fn sum(&self) -> u32 {
        self.items.iter().copied().sum()
    }

    fn last(&self) -> u32 {
        self.items[N - 1]
    }
}

fn main() {
    let encoder = HexPair;
    let encoded = encoder.encode(&[31_u8]);
    let window = FixedWindow {
        items: [3_u32, 5, 8, 13],
    };

    println!("hex = {}", std::str::from_utf8(&encoded).unwrap());
    println!("sum = {}", window.sum());
    println!("last = {}", window.last());
}

```

Exercises

Translate template-style helpers into Rust generics, simplify traits with associated types, and implement const-generic fixed-size types

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 19

Serialization and Data Contracts

Serde fundamentals, wire formats, schema evolution, custom serializers, zero-copy deserialization, and boundary contracts for distributed systems, WASM, and FFI

Overview

Serialization defines the contract between Rust services, files, queues, and external clients. This chapter covers versioned payloads, schema ownership, compatibility, and explicit transport boundaries.

Examples

Example 19-1. `serialization_contracts_versioned_event.rs` — *Serialization contracts versioned event*

```

use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize, PartialEq)]
#[serde(tag = "kind", rename_all = "snake_case")]
enum OrderEvent {
    Created {
        order_id: String,
        customer_id: String,
        total_cents: u64,
    },
    Cancelled {
        order_id: String,
        reason: Option<String>,
    },
}

#[derive(Debug, Serialize, Deserialize, PartialEq)]
struct EventEnvelope {
    schema_version: u16,
    #[serde(default)]
    trace_id: Option<String>,
    event_id: String,
    payload: OrderEvent,
}

fn main() {
    let envelope = EventEnvelope {
        schema_version: 2,
        trace_id: None,
        event_id: String::from("evt-100"),
        payload: OrderEvent::Created {
            order_id: String::from("ord-7"),
            customer_id: String::from("cust-9"),
            total_cents: 4200,
        },
    };

    let json = serde_json::to_string(&envelope).unwrap();
    let decoded: EventEnvelope = serde_json::from_str(&json).unwrap();

    let kind = match &decoded.payload {
        OrderEvent::Created { .. } => "created",
        OrderEvent::Cancelled { .. } => "cancelled",
    };

    let total_cents = match decoded.payload {
        OrderEvent::Created { total_cents, .. } => total_cents,
        OrderEvent::Cancelled { .. } => 0,
    };

    println!("schema = {}", decoded.schema_version);
    println!("kind = {}", kind);
    println!("total cents = {}", total_cents);
}

```

Example 19-2. `serialization_contracts_custom_zero_copy.rs` — *Serialization contracts custom zero copy*

```

use std::borrow::Cow;

use serde::{Deserialize, Deserializer, Serialize, Serializer};

fn cents_as_decimal<S>(value: &u64, serializer: S) -> Result<S::Ok, S::Error>
where
    S: Serializer,
{
    let text = format!("{:.{:02}}", value / 100, value % 100);
    serializer.serialize_str(&text)
}

fn decimal_as_cents<'de, D>(deserializer: D) -> Result<u64, D::Error>
where
    D: Deserializer<'de>,
{
    let text = Cow::<str>::deserialize(deserializer)?;
    let (units, cents) = text
        .split_once('.')
        .ok_or_else(|| serde::de::Error::custom("expected decimal amount"))?;

    let whole = units.parse::<u64>().map_err(serde::de::Error::custom)?;
    let frac = cents.parse::<u64>().map_err(serde::de::Error::custom)?;

    Ok(whole * 100 + frac)
}

#[derive(Debug, Serialize, Deserialize)]
struct BorrowedAudit<'a> {
    #[serde(borrow)]
    request_id: &'a str,
    #[serde(borrow)]
    route: Cow<'a, str>,
    #[serde(
        serialize_with = "cents_as_decimal",
        deserialize_with = "decimal_as_cents"
    )]
    amount_cents: u64,
}

fn main() {
    let raw = "{\"request_id\\\":\\\"req-7\\\",\\\"route\\\":\\\"/checkout\\\",\\\"amount_cents\\\":\\\"12.50\\\"}";
    let audit: BorrowedAudit<'_> = serde_json::from_str(raw).unwrap();

    println!("request = {}", audit.request_id);
    println!("route = {}", audit.route);
    println!("amount cents = {}", audit.amount_cents);
}

```

Exercises

Round-trip versioned events, add custom serializers, design zero-copy boundaries, and choose format strategies deliberately

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 20

Metaprogramming

Declarative and procedural macros, token streams, compile-time DSLs, code generation strategies, and when not to use macros

Overview

Declarative and procedural macros, token streams, compile-time DSLs, code generation strategies, and when not to use macros

Examples

Example 20-1. `metaprogramming_macro_rules_hygiene.rs` — *Metaprogramming macro rules hygiene*

```
macro_rules! add_one {
    ($value:expr) => {{
        let temp = $value;
        temp + 1
    }};
}

macro_rules! sum_values {
    ($($value:expr),* $(,)? ) => {{
        let mut total = 0;
        $(
            total += $value;
        )*
        total
    }};
}

fn main() {
    let total = 40;
    println!("next = {}", add_one!(total));
    println!("sum = {}", sum_values!(1, 2, 3));
    println!("outer total = {}", total);
}
```

Example 20-2. `metaprogramming_compile_time_dsl.rs` — *Metaprogramming compile time dsl*

```

#[derive(Debug)]
struct Route {
    method: &'static str,
    path: &'static str,
    auth: bool,
}

macro_rules! routes {
    ($( $( $method:ident $path:literal => $auth:ident ),* $(,)? ) => {{
        vec![
            $(
                Route {
                    method: stringify!($method),
                    path: $path,
                    auth: routes!(@auth $auth),
                }
            ),*
        ]
    }});
    (@auth public) => { false };
    (@auth private) => { true };
}

fn main() {
    let table = routes!(
        GET "/health" => public,
        POST "/orders" => private,
        DELETE "/orders/:id" => private,
    );

    let private_count = table.iter().filter(|route| route.auth).count();

    println!("routes = {}", table.len());
    println!("first = {} {}", table[0].method, table[0].path);
    println!("private = {}", private_count);
}

```

Exercises

Choose the right metaprogramming tool, write a small `macro_rules` helper, and sketch `derive` and `attribute` macro APIs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 21

Reflection and Type Introspection

Why Rust limits runtime reflection, Any and TypeId, downcasting, macro-based introspection, schema generation, and explicit metadata systems

Overview

Why Rust limits runtime reflection, Any and TypeId, downcasting, macro-based introspection, schema generation, and explicit metadata systems

Examples

Example 21-1. reflection_any_typeid_registry.rs — *Reflection any typeid registry*

```
use std::any::{Any, TypeId};
use std::collections::HashMap;

#[derive(Debug)]
struct RequestContext {
    trace_id: &'static str,
}

#[derive(Debug)]
struct RetryBudget(u32);

#[derive(Default)]
struct TypeMap {
    values: HashMap<TypeId, Box<dyn Any>>,
}

impl TypeMap {
    fn insert<T: 'static>(&mut self, value: T) {
        self.values.insert(TypeId::of::<T>(), Box::new(value));
    }

    fn get<T: 'static>(&self) -> Option<T> {
        self.values.get(&TypeId::of::<T>())?.downcast_ref::<T>()
    }

    fn contains<T: 'static>(&self) -> bool {
        self.values.contains_key(&TypeId::of::<T>())
    }
}

fn main() {
    let mut request_scoped = TypeMap::default();
    request_scoped.insert(RequestContext { trace_id: "req-7" });
    request_scoped.insert(RetryBudget(3));

    println!("has context = {}", request_scoped.contains::<RequestContext>());
    println!("retries = {}", request_scoped.get::<RetryBudget>().unwrap().0);
    println!("trace = {}", request_scoped.get::<RequestContext>().unwrap().trace_id);
}
```

Example 21-2. reflection_plugin_metadata_downcast.rs — *Reflection plugin metadata downcast*

```

use std::any::Any;

#[derive(Debug, Clone, Copy)]
struct PluginMetadata {
    name: &'static str,
    kind: &'static str,
    config_format: &'static str,
}

trait Plugin {
    fn metadata(&self) -> PluginMetadata;
    fn run(&self, input: &str) -> String;
    fn as_any(&self) -> &dyn Any;
}

struct JsonFormatter {
    pretty: bool,
}

struct RedactSecrets;

impl Plugin for JsonFormatter {
    fn metadata(&self) -> PluginMetadata {
        PluginMetadata {
            name: "json",
            kind: "formatter",
            config_format: "json",
        }
    }

    fn run(&self, input: &str) -> String {
        if self.pretty {
            format!("pretty({})", input)
        } else {
            format!("compact({})", input)
        }
    }

    fn as_any(&self) -> &dyn Any {
        self
    }
}

impl Plugin for RedactSecrets {
    fn metadata(&self) -> PluginMetadata {
        PluginMetadata {
            name: "redact",
            kind: "filter",
            config_format: "none",
        }
    }

    fn run(&self, input: &str) -> String {
        input.replace("token=", "token=***")
    }

    fn as_any(&self) -> &dyn Any {
        self
    }
}

fn main() {
    let plugins: Vec<Box<dyn Plugin>> = vec![

```

```
Box::new(JsonFormatter { pretty: true })),
Box::new(RedactSecrets),
];

let listed = plugins
    .iter()
    .map(|plugin| plugin.metadata().name)
    .collect::<Vec<_>>()
    .join(",");

let pretty = plugins
    .iter()
    .find_map(|plugin| plugin.as_any().downcast_ref::<JsonFormatter>())
    .map(|json| json.pretty)
    .unwrap_or(false);

println!("plugins = {}", listed);
println!("json pretty = {}", pretty);
println!("first kind = {}", plugins[0].metadata().kind);
}
```

Exercises

Build a small TypeId registry, downcast safely from erased values, and design explicit metadata for plugin boundaries

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 22

Multithreading in Rust

Rust's thread safety model, Send and Sync, spawning and scoped threads, shared state, channels, work stealing, and cross-language tradeoffs

Overview

Multithreaded Rust systems need clear ownership transfer, shared-state policy, and shutdown behavior. This chapter covers threads, scoped work, channels, and synchronization as production boundaries.

Examples

Example 22-1. `multithreading_owned_jobs_channel.rs` — *Multithreading owned jobs channel*

```

use std::collections::HashMap;
use std::sync::mpsc;
use std::thread;

#[derive(Debug)]
struct Job {
    name: &'static str,
    cost: u32,
}

fn spawn_worker(
    tx: mpsc::Sender<(&'static str, u32)>,
    worker: &'static str,
    jobs: Vec<Job>,
) -> thread::JoinHandle<()> {
    thread::spawn(move || {
        let total: u32 = jobs.iter().map(|job| job.cost).sum();
        tx.send((worker, total)).unwrap();
    })
}

fn main() {
    let (tx, rx) = mpsc::channel();

    let ingest_jobs = vec![
        Job { name: "parse", cost: 3 },
        Job { name: "validate", cost: 2 },
    ];
    let index_jobs = vec![
        Job { name: "index", cost: 4 },
        Job { name: "flush", cost: 1 },
    ];

    let ingest = spawn_worker(tx.clone(), "ingest", ingest_jobs);
    let index = spawn_worker(tx, "index", index_jobs);

    ingest.join().unwrap();
    index.join().unwrap();

    let mut totals = HashMap::new();
    for (worker, total) in rx {
        totals.insert(worker, total);
    }

    let grand: u32 = totals.values().copied().sum();

    println!("ingest total = {}", totals.get("ingest").copied().unwrap_or(0));
    println!("index total = {}", totals.get("index").copied().unwrap_or(0));
    println!("grand total = {}", grand);
}

```

Example 22-2. multithreading_shared_state_metrics.rs — *Multithreading shared state metrics*

```

use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counts = Arc::new(Mutex::new(HashMap::new(<'static str, usize>::new())));
    let mut handles = Vec::new();

    for route in ["api", "api", "billing", "api"] {
        let counts = Arc::clone(&counts);
        handles.push(thread::spawn(move || {
            let mut map = counts.lock().unwrap();
            *map.entry(route).or_insert(0) += 1;
        }));
    }

    for handle in handles {
        handle.join().unwrap();
    }

    let map = counts.lock().unwrap();
    println!("api = {}", map.get("api").copied().unwrap_or(0));
    println!("billing = {}", map.get("billing").copied().unwrap_or(0));
    println!("routes = {}", map.len());
}

```

Example 22-3. multithreading_scoped_threads_sum.rs — *Multithreading scoped threads sum*

```

use std::thread;

fn main() {
    let values = [2_u32, 4, 6, 8, 10, 12];
    let split_at = 3;

    thread::scope(|scope| {
        let (left, right) = values.split_at(split_at);

        let left_handle = scope.spawn(move || left.iter().copied().sum::<u32>());
        let right_handle = scope.spawn(move || right.iter().copied().sum::<u32>());

        let left_total = left_handle.join().unwrap();
        let right_total = right_handle.join().unwrap();

        println!("left = {}", left_total);
        println!("right = {}", right_total);
        println!("total = {}", left_total + right_total);
    });
}

```

Exercises

Classify Send and Sync, move owned data into worker threads, and compare channel-based and shared-state designs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 23

Synchronization Primitives

Mutex, RwLock, Condvar, atomics, memory ordering, barriers, channels, lock-free patterns, deadlock prevention, and choosing the right primitive

Overview

Mutex, RwLock, Condvar, atomics, memory ordering, barriers, channels, lock-free patterns, deadlock prevention, and choosing the right primitive

Examples

Example 23-1. `synchronization_mutex_condvar_queue.rs` — *Synchronization mutex condvar queue*

```

use std::collections::VecDeque;
use std::sync::{Arc, Condvar, Mutex};
use std::thread;

#[derive(Debug, Default)]
struct QueueState {
    jobs: VecDeque<&'static str>,
    closed: bool,
}

#[derive(Debug)]
struct SharedQueue {
    state: Mutex<QueueState>,
    ready: Condvar,
}

fn main() {
    let shared = Arc::new(SharedQueue {
        state: Mutex::new(QueueState::default()),
        ready: Condvar::new(),
    });

    let worker = {
        let shared = Arc::clone(&shared);
        thread::spawn(move || {
            let mut processed = 0;

            loop {
                let mut state = shared.state.lock().unwrap();

                while state.jobs.is_empty() && !state.closed {
                    state = shared.ready.wait(state).unwrap();
                }

                if state.jobs.is_empty() && state.closed {
                    break;
                }

                let _job = state.jobs.pop_front().unwrap();
                processed += 1;
            }

            processed
        })
    };

    {
        let mut state = shared.state.lock().unwrap();
        state.jobs.push_back("parse");
        state.jobs.push_back("flush");
        state.closed = true;
        shared.ready.notify_all();
    }

    let processed = worker.join().unwrap();
    let remaining = shared.state.lock().unwrap().jobs.len();

    println!("processed = {}", processed);
    println!("remaining = {}", remaining);
}

```

Example 23-2. `synchronization_rwlock_barrier.rs` — *Synchronization rwlock barrier*

```

use std::sync::{Arc, Barrier, RwLock};
use std::thread;

#[derive(Debug)]
struct Config {
    version: usize,
    mode: &'static str,
}

fn main() {
    let config = Arc::new(RwLock::new(Config {
        version: 1,
        mode: "steady",
    }));

    let start = Arc::new(Barrier::new(3));
    let mut handles = Vec::new();

    for _ in 0..2 {
        let config = Arc::clone(&config);
        let start = Arc::clone(&start);
        handles.push(thread::spawn(move || {
            start.wait();
            let snapshot = config.read().unwrap();
            snapshot.version
        }));
    }

    {
        let mut guard = config.write().unwrap();
        guard.version = 2;
        guard.mode = "burst";
    }

    start.wait();

    let reader_version_sum: usize = handles
        .into_iter()
        .map(|handle| handle.join().unwrap())
        .sum();
    let mode = config.read().unwrap().mode;

    println!("reader version sum = {}", reader_version_sum);
    println!("mode = {}", mode);
}

```

Example 23-3. `synchronization_atomics_ordering.rs` — *Synchronization atomics ordering*

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let value = AtomicUsize::new(0);
    let ready = AtomicBool::new(false);

    thread::scope(|scope| {
        scope.spawn(|| {
            value.store(42, Ordering::Relaxed);
            ready.store(true, Ordering::Release);
        });

        scope.spawn(|| {
            while !ready.load(Ordering::Acquire) {
                thread::yield_now();
            }

            println!("ready = {}", true);
            println!("value = {}", value.load(Ordering::Relaxed));
        });
    });
}
```

Exercises

Replace a Mutex with RwLock where appropriate, reason about Acquire and Release ordering, and repair deadlock-prone designs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 24

Coroutines, Futures, and Async Rust

Coroutines vs futures, async fn and await, compiler-generated state machines, Pin, executors, cancellation, structured concurrency, async traits, and lifetime pitfalls

Overview

Async Rust services depend on futures, executors, cancellation, and ownership across await points. This chapter defines the runtime contract behind coroutines and asynchronous state machines.

Examples

Example 24-1. `async_rust_async_fn_await_block_on.rs` — *Async rust async fn await block on*

```

use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

struct YieldOnce {
    value: &'static str,
    yielded: bool,
}

impl YieldOnce {
    fn new(value: &'static str) -> Self {
        Self {
            value,
            yielded: false,
        }
    }
}

impl Future for YieldOnce {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        if self.yielded {
            Poll::Ready(self.value)
        } else {
            self.yielded = true;
            Poll::Pending
        }
    }
}

async fn build_label(service: &'static str, route: &'static str) -> String {
    let left = YieldOnce::new(service).await;
    let right = YieldOnce::new(route).await;
    format!("{}", left, right)
}

struct NoopWake;

impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn block_on<F: Future>(future: F) -> F::Output {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut future = Box::pin(future);

    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Ready(value) => return value,
            Poll::Pending => println!("poll = pending"),
        }
    }
}

fn main() {
    let label = block_on(build_label("billing", "/ready"));
    println!("label = {}", label);
}

```

Example 24-2. `async_rust_manual_future_state_machine.rs` — *Async rust manual future state machine*

```

use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::task::{Context, Poll, Wake, Waker};

#[derive(Debug, Clone, Copy)]
enum Stage {
    Start,
    Waiting,
    Done,
}

struct Handshake {
    stage: Stage,
    remaining_polls: u8,
}

impl Future for Handshake {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<Self::Output> {
        match self.stage {
            Stage::Start => {
                self.stage = Stage::Waiting;
                Poll::Pending
            }
            Stage::Waiting if self.remaining_polls > 0 => {
                self.remaining_polls -= 1;
                Poll::Pending
            }
            Stage::Waiting => {
                self.stage = Stage::Done;
                Poll::Ready("connected")
            }
            Stage::Done => Poll::Ready("connected"),
        }
    }
}

struct NoopWake;

impl Wake for NoopWake {
    fn wake(self: Arc<Self>) {}
}

fn main() {
    let waker = Waker::from(Arc::new(NoopWake));
    let mut cx = Context::from_waker(&waker);
    let mut future = Box::pin(Handshake {
        stage: Stage::Start,
        remaining_polls: 1,
    });

    loop {
        match future.as_mut().poll(&mut cx) {
            Poll::Pending => {
                println!("pending stage = {:?}", future.as_ref().get_ref().stage);
                println!("pending retries = {}", future.as_ref().get_ref().remaining_polls);
            }
            Poll::Ready(value) => {
                println!("ready = {}", value);
                break;
            }
        }
    }
}

```

```
}  
}  
}
```

Exercises

Trace async state machines, repair Send-bound spawn problems, and model cancellation and cleanup explicitly

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 25

Tokio

Runtime architecture, tasks and scheduling, spawn vs spawn_blocking, timers, async TCP and UDP, async fs, channels, graceful shutdown, backpressure, and production Tokio patterns

Overview

Tokio provides the runtime layer for network services, timers, tasks, and blocking work isolation. This chapter covers backpressure, task ownership, shutdown, and runtime configuration.

Examples

Example 25-1. tokio_tasks_backpressure_spawn_blocking.rs — Tokio tasks backpressure spawn blocking

```
use tokio::sync::mpsc;
use tokio::time::{self, Duration};

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::(<Vec<u32>>(1));

    let producer = tokio::spawn(async move {
        tx.send(vec![1_u32, 2, 3]).await.unwrap();
        tx.send(vec![4_u32, 5]).await.unwrap();
    });

    let consumer = tokio::spawn(async move {
        let mut interval = time::interval(Duration::from_millis(10));
        let mut batches = 0_u32;
        let mut total = 0_u32;

        while let Some(batch) = rx.recv().await {
            interval.tick().await;

            let subtotal = tokio::task::spawn_blocking(move || batch.into_iter().sum::(<u32>()))
                .await
                .unwrap();

            total += subtotal;
            batches += 1;
        }

        (batches, total)
    });

    producer.await.unwrap();
    let (batches, total) = consumer.await.unwrap();

    println!("buffer = 1");
    println!("batches = {}", batches);
    println!("total = {}", total);
}
```

Example 25-2. tokio_tcp_graceful_shutdown.rs — Tokio tcp graceful shutdown

```

use tokio::io::{AsyncReadExt, AsyncWriteExt};
use tokio::sync::watch;

async fn handle(mut stream: tokio::net::TcpStream) -> std::io::Result<()> {
    stream.write_all(b"pong\n").await?;
    Ok(())
}

#[tokio::main]
async fn main() -> std::io::Result<()> {
    let listener = tokio::net::TcpListener::bind("127.0.0.1:0").await?;
    let addr = listener.local_addr()?;
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);

    let server = tokio::spawn(async move {
        let mut accepted = 0_usize;

        loop {
            tokio::select! {
                _ = shutdown_rx.changed() => {
                    if *shutdown_rx.borrow() {
                        break accepted;
                    }
                }
                Ok((stream, _peer)) = listener.accept() => {
                    accepted += 1;
                    tokio::spawn(handle(stream));
                }
            }
        }
    });

    let client_a = tokio::spawn(async move {
        let mut stream = tokio::net::TcpStream::connect(addr).await.unwrap();
        let mut buf = [0_u8; 5];
        stream.read_exact(&mut buf).await.unwrap();
        String::from_utf8_lossy(&buf).trim().to_string()
    });

    let client_b = tokio::spawn(async move {
        let mut stream = tokio::net::TcpStream::connect(addr).await.unwrap();
        let mut buf = [0_u8; 5];
        stream.read_exact(&mut buf).await.unwrap();
        String::from_utf8_lossy(&buf).trim().to_string()
    });

    let a = client_a.await.unwrap();
    let b = client_b.await.unwrap();

    shutdown_tx.send(true).unwrap();
    let accepted = server.await.unwrap();

    println!("accepted = {}", accepted);
    println!("client_a = {}", a);
    println!("client_b = {}", b);
    Ok(())
}

```

Exercises

Build a small Tokio TCP service, add graceful shutdown, and move blocking CPU work behind `spawn_blocking`

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 26

Task Libraries and Parallel Execution

Tokio tasks, Rayon, Crossbeam, futures orchestration, CPU vs IO workloads, thread pools, backpressure, cancellation, retry, and task API design

Overview

Tokio tasks, Rayon, Crossbeam, futures orchestration, CPU vs IO workloads, thread pools, backpressure, cancellation, retry, and task API design

Examples

Example 26-1. `task_libraries_tokio_orchestration.rs` — *Task libraries tokio orchestration*

```

use tokio::sync::{mpsc, watch};
use tokio::task::JoinSet;

#[derive(Debug, Clone, Copy)]
struct Job {
    id: u32,
    needs_retry: bool,
}

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<Job>(2);
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);

    let producer = tokio::spawn(async move {
        tx.send(Job { id: 1, needs_retry: false }).await.unwrap();
        tx.send(Job { id: 2, needs_retry: true }).await.unwrap();
        tx.send(Job { id: 3, needs_retry: false }).await.unwrap();
    });

    let worker = tokio::spawn(async move {
        let mut set = JoinSet::new();
        let mut completed = 0_u32;
        let mut retries = 0_u32;

        while let Some(job) = rx.recv().await {
            set.spawn(async move {
                if job.needs_retry {
                    Err(job.id)
                } else {
                    Ok(job.id)
                }
            });
        }

        while let Some(result) = set.join_next().await {
            match result.unwrap() {
                Ok(_id) => completed += 1,
                Err(_id) => {
                    retries += 1;
                    completed += 1;
                }
            }
        }

        shutdown_rx.changed().await.unwrap();
        (completed, retries, *shutdown_rx.borrow())
    });

    producer.await.unwrap();
    shutdown_tx.send(true).unwrap();
    let (completed, retries, cancelled) = worker.await.unwrap();

    println!("buffer = 2");
    println!("completed = {}", completed);
    println!("retries = {}", retries);
    println!("cancelled = {}", cancelled);
}

```

Example 26-2. `task_libraries_rayon_crossbeam.rs` — *Task libraries rayon crossbeam*

```

use rayon::prelude::*;

fn main() {
    let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(2)
        .build()
        .unwrap();

    let (tx, rx) = crossbeam::channel::bounded::(<Vec<u64>>(2));

    tx.send(vec![1_u64, 2, 3, 4]).unwrap();
    tx.send(vec![5_u64, 6, 7, 8]).unwrap();
    tx.send(vec![9_u64, 10]).unwrap();
    drop(tx);

    let (batches, total) = pool.install(|| {
        let mut batches = 0_u64;
        let mut total = 0_u64;

        while let Ok(batch) = rx.recv() {
            let subtotal: u64 = batch
                .par_iter()
                .copied()
                .map(|value| value * 2)
                .sum();

            total += subtotal;
            batches += 1;
        }

        (batches, total)
    });

    println!("batches = {}", batches);
    println!("scaled total = {}", total);
    println!("pool threads = {}", 2);
}

```

Exercises

Choose Tokio, Rayon, Crossbeam, or futures utilities; implement a bounded worker queue; and add cancellation and retry policy to task orchestration

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 27

IO Tricks and Systems Programming Patterns

Buffered IO, zero-copy concepts, memory-mapped files, scatter/gather IO, blocking vs async IO, descriptor ownership, socket tuning, backpressure, and IO profiling

Overview

Buffered IO, zero-copy concepts, memory-mapped files, scatter/gather IO, blocking vs async IO, descriptor ownership, socket tuning, backpressure, and IO profiling

Examples

Example 27-1. `io_patterns_buffered_backpressure.rs` — *io patterns buffered backpressure*

```

use std::io::{self, BufRead, BufReader, BufWriter, Cursor, Write};
use std::sync::mpsc;
use std::thread;

fn main() -> io::Result<()> {
    let input = Cursor::new("alpha\\nbeta\\ngamma\\n".as_bytes());
    let mut reader = BufReader::new(input);
    let (tx, rx) = mpsc::sync_channel::<String>(1);

    let producer = thread::spawn(move || {
        let mut line = String::new();
        let mut sent = 0usize;

        loop {
            line.clear();
            let read = reader.read_line(&mut line).unwrap();
            if read == 0 {
                break;
            }

            tx.send(line.trim_end().to_string()).unwrap();
            sent += 1;
        }

        sent
    });

    let mut out = Vec::new();
    let mut received = 0usize;
    let mut batches = 0usize;
    let mut pending = Vec::with_capacity(2);

    {
        let mut writer = BufWriter::new(&mut out);

        while let Ok(line) = rx.recv() {
            pending.push(line);

            if pending.len() == 2 {
                for item in pending.drain(..) {
                    writeln!(writer, "{}", item)?;
                }
                batches += 1;
            }

            received += 1;
        }

        if !pending.is_empty() {
            for item in pending.drain(..) {
                writeln!(writer, "{}", item)?;
            }
            batches += 1;
        }

        writer.flush()?;
    }

    println!("sent = {}", producer.join().unwrap());
    println!("received = {}", received);
    println!("batches = {}", batches);
    println!("bytes = {}", out.len());
}

```



```
Ok(())
}
```

Example 27-2. `io_patterns_scatter_gather_socket.rs` — *io patterns scatter gather socket*

```
use std::io::{self, IoSlice, Read, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

fn write_all_vectored(stream: &mut TcpStream, mut parts: &mut [IoSlice<'_>]) -> io::Result<()> {
    while !parts.is_empty() {
        let wrote = stream.write_vectored(parts)?;
        if wrote == 0 {
            return Err(io::Error::new(
                io::ErrorKind::WriteZero,
                "socket closed before full response",
            ));
        }
        IoSlice::advance_slices(&mut parts, wrote);
    }
}

Ok(())
}

fn main() -> io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:0"?);
    let addr = listener.local_addr()?;

    let server = thread::spawn(move || -> io::Result<usize> {
        let (mut stream, _) = listener.accept()?;
        stream.set_nodelay(true)?;

        let mut parts = [IoSlice::new(b"hdr:"), IoSlice::new(b"payload")];
        write_all_vectored(&mut stream, &mut parts)?;
        stream.flush()?;
        Ok(parts.len())
    });

    let mut client = TcpStream::connect(addr)?;
    let mut buf = [0_u8; 11];
    client.read_exact(&mut buf)?;

    let parts = server.join().unwrap()?;
    println!("nodelay = {}", true);
    println!("vectored parts = {}", parts);
    println!("client = {}", String::from_utf8_lossy(&buf));
    Ok(())
}
```

Exercises

Compare buffered and unbuffered reads, design a backpressure-aware IO pipeline, and reason clearly about descriptor ownership

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 28

C++ Integration

Rust FFI fundamentals, C ABI seams, ownership, status-code error handling, unwind policy, and interop testing

Overview

Native-library integration is a production boundary with ABI, ownership, memory layout, unwind, and error translation requirements. This chapter defines Rust wrapper patterns for that boundary.

Examples

Example 28-1. `cpp_integration_calling_c_abi.rs` — *Cpp integration calling c abi*

```
mod c_shim {
    #[unsafe(no_mangle)]
    pub extern "C" fn ffi_demo_abs(input: i32) -> i32 {
        input.abs()
    }
}

unsafe extern "C" {
    fn ffi_demo_abs(input: i32) -> i32;
}

fn safe_abs(input: i32) -> i32 {
    unsafe {
        // SAFETY:
        // - ffi_demo_abs uses the C ABI.
        // - the function takes a plain i32 and returns a plain i32.
        // - there is no cross-language ownership transfer in this call.
        ffi_demo_abs(input)
    }
}

fn main() {
    let left = -7_i32;
    let right = 11_i32;

    println!("abs({}) = {}", left, safe_abs(left));
    println!("abs({}) = {}", right, safe_abs(right));
}
```

Example 28-2. `cpp_integration_export_rust_c_abi.rs` — *Cpp integration export rust c abi*

```

#[unsafe(no_mangle)]
pub extern "C" fn sum_i32s(ptr: *const i32, len: usize, out_total: *mut i64) -> i32 {
    if out_total.is_null() {
        return 1;
    }

    if ptr.is_null() && len != 0 {
        return 2;
    }

    let slice = unsafe {
        // SAFETY:
        // - ptr is either non-null for len elements, or len == 0.
        // - the caller promises the memory is readable for len i32 values.
        std::slice::from_raw_parts(ptr, len)
    };

    let total = slice.iter().map(|&value| value as i64).sum::<i64>();

    unsafe {
        // SAFETY:
        // - out_total was checked for null above.
        // - the caller promises it points to writable i64 storage.
        *out_total = total;
    }

    0
}

fn main() {
    let values = [3_i32, 4, 5];
    let mut total = -1_i64;

    let status = sum_i32s(values.as_ptr(), values.len(), &mut total);

    println!("status = {}", status);
    println!("total = {}", total);
}

```

Exercises

Flatten Rust types into a C ABI-safe surface, map ownership across the seam, and choose an interop strategy deliberately

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 29

JavaScript and C++ Integration for WASM

Rust to WebAssembly, wasm-bindgen, JS interop, boundary data choices, C++ shims, WASI, performance limits, serialization, and debugging

Overview

WebAssembly integration needs explicit host calls, memory ownership, data batching, and ABI rules. This chapter defines the boundary contracts for browser-facing modules and native shims.

Examples

Example 29-1. `wasm_bindgen_string_array_boundary.rs` — *Wasm bindgen string array boundary*

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn build_label(service: &str, route: &str) -> String {
    format!("{:}:{:}", service, route)
}

#[wasm_bindgen]
pub fn sum_bytes(bytes: &[u8]) -> u32 {
    bytes.iter().map(|&value| value as u32).sum()
}

fn main() {
    let label = build_label("billing", "/ready");
    let total = sum_bytes(&[1_u8, 2, 3, 4]);

    println!("label = {}", label);
    println!("sum = {}", total);
}
```

Example 29-2. `wasm_cpp_ffi_boundary.rs` — *Wasm cpp ffi boundary*

```

mod cpp_shim {
    #[unsafe(no_mangle)]
    pub extern "C" fn cpp_route_score(ptr: *const u8, len: usize) -> u32 {
        if ptr.is_null() && len != 0 {
            return 0;
        }

        let bytes = unsafe {
            // SAFETY:
            // - ptr is either null with len == 0 or valid for len bytes.
            // - the shim only reads the bytes for the duration of the call.
            std::slice::from_raw_parts(ptr, len)
        };

        (bytes.len() as u32) * 10
    }
}

unsafe extern "C" {
    fn cpp_route_score(ptr: *const u8, len: usize) -> u32;
}

pub fn route_score(route: &str) -> u32 {
    unsafe {
        // SAFETY:
        // - route.as_ptr() is valid for route.len() bytes for the duration of the call.
        // - the shim borrows the bytes only for the duration of the call.
        cpp_route_score(route.as_ptr(), route.len())
    }
}

fn main() {
    let route = "/orders";
    println!("route = {}", route);
    println!("score = {}", route_score(route));
}

```

Exercises

Export Rust to JavaScript with `wasm-bindgen`, choose serialization and memory boundaries, and reason about JS/WASM copying costs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 30

AMQP and Message Brokers

Broker fundamentals, AMQP exchanges and queues, RabbitMQ with Rust, acknowledgments, retries, dead-letter queues, idempotency, backpressure, serialization, and observability

Overview

Message-broker systems need reliable routing, redelivery handling, idempotent consumers, dead-letter policy, and local backpressure. This chapter covers AMQP as an operational contract.

Examples

Example 30-1. `amqp_direct_exchange_routing.rs` — *Amqp direct exchange routing*

```
use std::collections::HashMap;

#[derive(Debug, Default)]
struct DirectExchange {
    bindings: HashMap<String, Vec<String>>,
}

impl DirectExchange {
    fn bind(&mut self, routing_key: &str, queue: &str) {
        self.bindings
            .entry(routing_key.to_string())
            .or_default()
            .push(queue.to_string());
    }

    fn route(&self, routing_key: &str) -> Vec<String> {
        self.bindings
            .get(routing_key)
            .cloned()
            .unwrap_or_default()
    }
}

fn main() {
    let mut exchange = DirectExchange::default();

    exchange.bind("orders.created", "billing");
    exchange.bind("orders.created", "search");
    exchange.bind("orders.cancelled", "billing");

    let created = exchange.route("orders.created");
    let cancelled = exchange.route("orders.cancelled");

    println!("created = {}", created.join(", "));
    println!("cancelled = {}", cancelled.join(", "));
    println!("unrouted = {}", exchange.route("orders.refunded").len());
}
```

Example 30-2. `amqp_idempotent_consumer.rs` — *Amqp idempotent consumer*

```

use std::collections::{HashSet, VecDeque};

#[derive(Debug, Clone)]
struct Delivery {
    message_id: &'static str,
    order_id: &'static str,
    attempt: u8,
}

#[derive(Default)]
struct Consumer {
    processed_ids: HashSet<String>,
    retry_queue: VecDeque<Delivery>,
    dlq: Vec<Delivery>,
    processed: usize,
    duplicates: usize,
    retried: usize,
}

impl Consumer {
    fn on_delivery(&mut self, delivery: Delivery) {
        if self.processed_ids.contains(delivery.message_id) {
            self.duplicates += 1;
            return;
        }

        if delivery.order_id == "ord-fail" {
            if delivery.attempt < 3 {
                self.retried += 1;
                self.retry_queue.push_back(Delivery {
                    attempt: delivery.attempt + 1,
                    ..delivery
                });
            } else {
                self.dlq.push(delivery);
            }
            return;
        }

        self.processed += 1;
        self.processed_ids.insert(delivery.message_id.to_string());
    }
}

fn main() {
    let mut consumer = Consumer::default();

    consumer.on_delivery(Delivery {
        message_id: "msg-1",
        order_id: "ord-1",
        attempt: 1,
    });
    consumer.on_delivery(Delivery {
        message_id: "msg-1",
        order_id: "ord-1",
        attempt: 1,
    });
    consumer.on_delivery(Delivery {
        message_id: "msg-2",
        order_id: "ord-fail",
        attempt: 1,
    });
}

```

```
while let Some(delivery) = consumer.retry_queue.pop_front() {
    consumer.on_delivery(delivery);
}

consumer.on_delivery(Delivery {
    message_id: "msg-3",
    order_id: "ord-3",
    attempt: 1,
});

println!("processed = {}", consumer.processed);
println!("duplicates = {}", consumer.duplicates);
println!("retried = {}", consumer.retried);
println!("dlq = {}", consumer.dlq.len());
}
```

Exercises

Model an idempotent consumer, add retry and dead-letter handling, and design versioned message contracts

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 31

Distributed Task Execution

Task queues, work distribution models, delivery semantics, leasing, retries, failure isolation, result aggregation, task graphs, tracing, and profiling for distributed workloads

Overview

Task queues, work distribution models, delivery semantics, leasing, retries, failure isolation, result aggregation, task graphs, tracing, and profiling for distributed workloads

Examples

Example 31-1. distributed_tasks_lease_idempotent.rs — Distributed tasks lease idempotent

```

use std::collections::{HashMap, HashSet, VecDeque};

#[derive(Debug, Clone)]
struct Task {
    id: &'static str,
    payload: &'static str,
}

#[derive(Debug)]
struct Lease {
    task: Task,
    deadline: u64,
}

struct LeaseQueue {
    visible: VecDeque<Task>,
    leased: HashMap<&'static str, Lease>,
    completed: HashSet<&'static str>,
    visibility_timeout: u64,
    now: u64,
}

impl LeaseQueue {
    fn new(visibility_timeout: u64) -> Self {
        Self {
            visible: VecDeque::new(),
            leased: HashMap::new(),
            completed: HashSet::new(),
            visibility_timeout,
            now: 0,
        }
    }

    fn push(&mut self, id: &'static str, payload: &'static str) {
        self.visible.push_back(Task { id, payload });
    }

    fn claim(&mut self, worker: &'static str) -> Option<(&'static str, &'static str, &'static str)> {
        let task = self.visible.pop_front()?;
        let deadline = self.now + self.visibility_timeout;
        let task_id = task.id;
        let payload = task.payload;
        self.leased.insert(task_id, Lease { task, deadline });
        Some((worker, task_id, payload))
    }

    fn advance_time(&mut self, delta: u64) {
        self.now += delta;
    }

    fn requeue_expired(&mut self) {
        let expired: Vec<&'static str> = self
            .leased
            .iter()
            .filter(|(&task_id, lease)| lease.deadline <= self.now)
            .map(|(&task_id, _)| task_id)
            .collect();

        for task_id in expired {
            let lease = self.leased.remove(task_id).unwrap();
            self.visible.push_back(lease.task);
        }
    }
}

```

```
fn finish_once(&mut self, task_id: &'static str) -> bool {
    if !self.completed.insert(task_id) {
        return false;
    }

    self.leased.remove(task_id);
    true
}

fn main() {
    let mut queue = LeaseQueue::new(5);
    queue.push("task-1", "rebuild-index");

    let first = queue.claim("worker-a").unwrap();
    queue.advance_time(6);
    queue.requeue_expired();

    let redelivery = queue.claim("worker-b").unwrap();
    let _applied = queue.finish_once(redelivery.1);
    let duplicate = !queue.finish_once(redelivery.1);

    println!("claimed = {}", first.1);
    println!("redelivered = {}", redelivery.1);
    println!("completed = {}", queue.completed.len());
    println!("duplicates ignored = {}", duplicate);
}
```

Example 31-2. distributed_tasks_graph_trace.rs — *Distributed tasks graph trace*

```

use std::collections::BTreeSet;

type TaskId = usize;

#[derive(Debug)]
struct TaskSpec {
    name: &'static str,
    deps: Vec<TaskId>,
    output: u32,
}

#[derive(Default)]
struct TaskGraph {
    nodes: Vec<TaskSpec>,
}

impl TaskGraph {
    fn add(&mut self, name: &'static str, deps: Vec<TaskId>, output: u32) -> TaskId {
        let id = self.nodes.len();
        self.nodes.push(TaskSpec { name, deps, output });
        id
    }

    fn ready(&self, done: &BTreeSet<TaskId>) -> Vec<TaskId> {
        self.nodes
            .iter()
            .enumerate()
            .filter(|(id, node)| !done.contains(id) && node.deps.iter().all(|dep| done.contains(dep)))
            .map(|(id, _)| id)
            .collect()
    }
}

fn main() {
    let mut graph = TaskGraph::default();
    let fetch = graph.add("fetch", vec![], 5);
    let parse = graph.add("parse", vec![fetch], 8);
    let enrich = graph.add("enrich", vec![parse], 13);
    let store = graph.add("store", vec![parse], 3);
    let _notify = graph.add("notify", vec![enrich, store], 2);

    let trace_id = "trace-7";
    let mut done = BTreeSet::new();
    let mut order = Vec::new();
    let mut aggregate = 0_u32;

    while done.len() < graph.nodes.len() {
        let ready = graph.ready(&done);

        for id in ready {
            let node = &graph.nodes[id];
            aggregate += node.output;
            order.push(node.name);
            done.insert(id);
        }
    }

    println!("completed = {}", order.len());
    println!("aggregate = {}", aggregate);
    println!("final = {}", order.last().copied().unwrap_or("none"));
}

```

```
println!("trace = {}", trace_id);  
}
```

Exercises

Design a lease-based distributed worker, model at-least-once execution with idempotency, and trace distributed task graphs end to end

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 32

MPI and High-Performance Computing

MPI process model, collectives, data layout, Rust MPI crates, hybrid MPI plus threads, profiling, and HPC production environments

Overview

MPI process model, collectives, data layout, Rust MPI crates, hybrid MPI plus threads, profiling, and HPC production environments

Examples

Example 32-1. `mpi_partition_dense_rows.rs` — *Mpi partition dense rows*

```
#[derive(Debug, Clone, Copy)]
struct Partition {
    start_row: usize,
    end_row: usize,
}

fn block_range(rows: usize, ranks: usize, rank: usize) -> Partition {
    let base = rows / ranks;
    let remainder = rows % ranks;
    let start = rank * base + rank.min(remainder);
    let len = base + usize::from(rank < remainder);

    Partition {
        start_row: start,
        end_row: start + len,
    }
}

fn local_sum(matrix: &[f64], cols: usize, part: Partition) -> f64 {
    let start = part.start_row * cols;
    let end = part.end_row * cols;
    matrix[start..end].iter().copied().sum()
}

fn main() {
    let rows = 8_usize;
    let cols = 3_usize;
    let ranks = 3_usize;
    let rank = 1_usize;

    let matrix: Vec<f64> = (1..=(rows * cols)).map(|value| value as f64).collect();
    let part = block_range(rows, ranks, rank);
    let subtotal = local_sum(&matrix, cols, part);

    println!("rank = {}", rank);
    println!("rows = {}..{}", part.start_row, part.end_row);
    println!("local rows = {}", part.end_row - part.start_row);
    println!("subtotal = {:.1}", subtotal);
}
```

Example 32-2. `mpi_collective_counts_and_allreduce.rs` — *Mpi collective counts and allreduce*

```

fn row_counts(rows: usize, ranks: usize) -> Vec<usize> {
    let base = rows / ranks;
    let remainder = rows % ranks;

    (0..ranks)
        .map(|rank| base + usize::from(rank < remainder))
        .collect()
}

fn displacements_in_cells(counts: &[usize], cols: usize) -> Vec<usize> {
    let mut out = Vec::with_capacity(counts.len());
    let mut offset = 0_usize;

    for &rows_for_rank in counts {
        out.push(offset);
        offset += rows_for_rank * cols;
    }

    out
}

fn main() {
    let rows = 10_usize;
    let cols = 4_usize;
    let ranks = 3_usize;
    let rank = 1_usize;

    let counts = row_counts(rows, ranks);
    let displs = displacements_in_cells(&counts, cols);
    let local_norms = [7_u64, 9, 5];
    let allreduce_sum: u64 = local_norms.iter().copied().sum();

    println!("counts = {:?}", counts);
    println!("displs = {:?}", displs);
    println!("send cells = {}", counts[rank] * cols);
    println!("allreduce = {}", allreduce_sum);
}

```

Exercises

Partition matrix rows across ranks, choose collective-friendly layouts, and reason about hybrid MPI plus threads and communication profiling

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 33

Performance-Oriented Rust

Rust's performance model, allocation awareness, cache locality, branch prediction, static dispatch, iterator tradeoffs, benchmarking methodology, and release profile choices

Overview

Performance work starts with allocation, data movement, cache behavior, and measurement. This chapter turns Rust optimization into a repeatable review process tied to workload evidence.

Examples

Example 33-1. `performance_allocation_borrowed_filter.rs` — *Performance allocation borrowed filter*

```
#[derive(Debug)]
struct Request<'a> {
    route: &'a str,
    bytes: usize,
}

fn hot_routes<'a>(requests: &'a [Request<'a>], min_bytes: usize) -> Vec<&'a str> {
    let mut out = Vec::with_capacity(requests.len());

    for request in requests {
        if request.bytes >= min_bytes {
            out.push(request.route);
        }
    }

    out
}

fn main() {
    let requests = [
        Request {
            route: "/health",
            bytes: 128,
        },
        Request {
            route: "/search",
            bytes: 900,
        },
        Request {
            route: "/checkout",
            bytes: 512,
        },
        Request {
            route: "/metrics",
            bytes: 64,
        },
    ];

    let hot = hot_routes(&requests, 512);

    println!("hot = {}", hot.len());
    println!("first = {}", hot.first().copied().unwrap_or("none"));
    println!("capacity ok = {}", hot.len() <= hot.capacity());
}
```


Example 33-2. `performance_row_major_scan.rs` — *Performance row major scan*

```

#[derive(Debug)]
struct Grid {
    rows: usize,
    cols: usize,
    data: Vec<u32>,
}

impl Grid {
    fn row_sums(&self) -> Vec<u32> {
        self.data
            .chunks(self.cols)
            .map(|row| row.iter().copied().sum())
            .collect()
    }

    fn total(&self) -> u32 {
        self.data.iter().copied().sum()
    }
}

fn main() {
    let grid = Grid {
        rows: 2,
        cols: 4,
        data: vec![1_u32, 2, 3, 4, 5, 6, 7, 8],
    };

    let sums = grid.row_sums();

    println!("row0 = {}", sums[0]);
    println!("row1 = {}", sums[1]);
    println!("total = {}", grid.total());
}

```

Exercises

Write a benchmark plan, refactor for locality, compare loops and iterators responsibly, and make measurement discipline explicit

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 34

Memory Profiling

Measuring allocations, heap profiling, clone pressure, cycle leaks, fragmentation, arena retention, async memory usage, allocator behavior, and footprint reduction

Overview

Measuring allocations, heap profiling, clone pressure, cycle leaks, fragmentation, arena retention, async memory usage, allocator behavior, and footprint reduction

Examples

Example 34-1. `memory_profiling_clone_pressure_counter.rs` — *Memory profiling clone pressure counter*

```
#[derive(Debug, Default)]
struct CloneStats {
    clones: usize,
    cloned_bytes: usize,
}

fn select_api_routes(routes: &[&str], stats: &mut CloneStats) -> Vec<String> {
    let mut out = Vec::with_capacity(routes.len());

    for &route in routes {
        if route.starts_with("/api/") {
            stats.clones += 1;
            stats.cloned_bytes += route.len();
            out.push(route.to_string());
        }
    }

    out
}

fn main() {
    let routes = ["/api/orders", "/health", "/api/users"];
    let mut stats = CloneStats::default();
    let selected = select_api_routes(&routes, &mut stats);

    println!("selected = {}", selected.len());
    println!("clones = {}", stats.clones);
    println!("cloned bytes = {}", stats.cloned_bytes);
}
```

Example 34-2. `memory_profiling_rc_cycle_leak.rs` — *Memory profiling rc cycle leak*

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct BadNode {
    name: &'static str,
    parent: RefCell<Option<Rc<BadNode>>>,
    children: RefCell<Vec<Rc<BadNode>>>,
}

#[derive(Debug)]
struct GoodNode {
    name: &'static str,
    parent: RefCell<Weak<GoodNode>>,
    children: RefCell<Vec<Rc<GoodNode>>>,
}

fn main() {
    let bad_root = Rc::new(BadNode {
        name: "root",
        parent: RefCell::new(None),
        children: RefCell::new(Vec::new()),
    });
    let bad_leaf = Rc::new(BadNode {
        name: "leaf",
        parent: RefCell::new(None),
        children: RefCell::new(Vec::new()),
    });

    bad_root.children.borrow_mut().push(Rc::clone(&bad_leaf));
    *bad_leaf.parent.borrow_mut() = Some(Rc::clone(&bad_root));

    let good_root = Rc::new(GoodNode {
        name: "root",
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });
    let good_leaf = Rc::new(GoodNode {
        name: "leaf",
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });

    good_root.children.borrow_mut().push(Rc::clone(&good_leaf));
    *good_leaf.parent.borrow_mut() = Rc::downgrade(&good_root);

    println!("bad strong = {}", Rc::strong_count(&bad_root));
    println!("good strong = {}", Rc::strong_count(&good_root));
    println!(
        "good parent = {}",
        good_leaf
            .parent
            .borrow()
            .upgrade()
            .map(|node| node.name)
            .unwrap_or("none")
    );
}

```

Exercises

Find clone pressure, diagnose Rc and Arc leaks, choose profiling tools, and build a memory profiling checklist

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 35

Performance Profiling

CPU profiling, flame graphs, sampling vs instrumentation, Criterion benchmarks, async and lock profiling, serialization and IO analysis, and WASM and FFI profiling boundaries

Overview

Performance profiling converts slow paths into verifiable bottlenecks with owners, budgets, and measurements. This chapter covers benchmarks, flame graphs, counters, and regression checks.

Examples

Example 35-1. `performance_profiling_hot_stage_summary.rs` — *Performance profiling hot stage summary*

```
[derive(Debug, Clone, Copy)]
struct StageSample {
    name: &'static str,
    micros: u64,
}

fn hottest_stage(samples: &[StageSample]) -> (&'static str, u64, u64) {
    let total: u64 = samples.iter().map(|sample| sample.micros).sum();
    let hottest = samples
        .iter()
        .max_by_key(|sample| sample.micros)
        .copied()
        .unwrap();

    (hottest.name, hottest.micros, total)
}

fn main() {
    let samples = [
        StageSample {
            name: "parse",
            micros: 180,
        },
        StageSample {
            name: "serialize",
            micros: 620,
        },
        StageSample {
            name: "lock_wait",
            micros: 40,
        },
    ];

    let (stage, hottest, total) = hottest_stage(&samples);

    println!("hottest = {}", stage);
    println!("stage us = {}", hottest);
    println!("total us = {}", total);
}
```

Example 35-2. `performance_profiling_pipeline_bottleneck.rs` — *Performance profiling pipeline bottleneck*

```
#[derive(Debug)]
struct PipelineStats {
    cpu_us: u64,
    io_wait_us: u64,
    lock_wait_us: u64,
    serialize_us: u64,
    serialized_bytes: usize,
}

fn dominant(stats: &PipelineStats) -> &'static str {
    [
        ("cpu", stats.cpu_us),
        ("io", stats.io_wait_us),
        ("lock", stats.lock_wait_us),
        ("serialize", stats.serialize_us),
    ]
    .into_iter()
    .max_by_key(|(_, value)| *value)
    .map(|(name, _)| name)
    .unwrap()
}

fn wall_time(stats: &PipelineStats) -> u64 {
    stats.cpu_us + stats.io_wait_us + stats.lock_wait_us + stats.serialize_us
}

fn main() {
    let stats = PipelineStats {
        cpu_us: 410,
        io_wait_us: 120,
        lock_wait_us: 90,
        serialize_us: 180,
        serialized_bytes: 16_384,
    };

    println!("dominant = {}", dominant(&stats));
    println!("serialized bytes = {}", stats.serialized_bytes);
    println!("wall us = {}", wall_time(&stats));
}
```

Exercises

Interpret flame graphs, design Criterion benchmarks, separate CPU and waiting bottlenecks, and profile async, WASM, and FFI boundaries deliberately

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 36

Distributed Tasks Profiling

End-to-end latency, queue latency, worker saturation, retry storms, tail latency, distributed tracing, metrics design, task-graph profiling, and capacity planning

Overview

End-to-end latency, queue latency, worker saturation, retry storms, tail latency, distributed tracing, metrics design, task-graph profiling, and capacity planning

Examples

Example 36-1. distributed_profiling_latency_window.rs — Distributed profiling latency window

```
[derive(Debug, Clone, Copy)]
struct WindowStats {
    claimed: u32,
    completed: u32,
    retried: u32,
    queue_p95_ms: u64,
    run_p95_ms: u64,
    busy_workers: u32,
    total_workers: u32,
}

fn end_to_end_p95_ms(stats: &WindowStats) -> u64 {
    stats.queue_p95_ms + stats.run_p95_ms
}

fn worker_saturation(stats: &WindowStats) -> f64 {
    stats.busy_workers as f64 / stats.total_workers as f64
}

fn retry_rate(stats: &WindowStats) -> f64 {
    stats.retried as f64 / stats.claimed as f64
}

fn main() {
    let stats = WindowStats {
        claimed: 120,
        completed: 110,
        retried: 18,
        queue_p95_ms: 140,
        run_p95_ms: 320,
        busy_workers: 17,
        total_workers: 20,
    };

    println!("e2e p95 = {}", end_to_end_p95_ms(&stats));
    println!("queue p95 = {}", stats.queue_p95_ms);
    println!("worker saturation = {:.2}", worker_saturation(&stats));
    println!("retry rate = {:.2}", retry_rate(&stats));
}
```

Example 36-2. distributed_profiling_task_graph.rs — Distributed profiling task graph

```

type StageId = usize;

#[derive(Debug)]
struct Stage {
    name: &'static str,
    deps: Vec<StageId>,
    queue_ms: u64,
    run_ms: u64,
}

fn critical_path(stages: &[Stage]) -> (u64, &'static str) {
    let mut totals = vec![0_u64; stages.len()];
    let mut best = 0_u64;
    let mut tail = "none";

    for (id, stage) in stages.iter().enumerate() {
        let upstream = stage
            .deps
            .iter()
            .map(|&dep| totals[dep])
            .max()
            .unwrap_or(0);

        totals[id] = upstream + stage.queue_ms + stage.run_ms;

        if totals[id] >= best {
            best = totals[id];
            tail = stage.name;
        }
    }

    (best, tail)
}

fn main() {
    let stages = vec![
        Stage {
            name: "fetch",
            deps: vec![],
            queue_ms: 20,
            run_ms: 70,
        },
        Stage {
            name: "parse",
            deps: vec![0],
            queue_ms: 30,
            run_ms: 90,
        },
        Stage {
            name: "enrich",
            deps: vec![1],
            queue_ms: 40,
            run_ms: 120,
        },
        Stage {
            name: "store",
            deps: vec![1],
            queue_ms: 10,
            run_ms: 60,
        },
        Stage {
            name: "notify",
            deps: vec![2, 3],

```



```
        queue_ms: 15,
        run_ms: 30,
    },
];

let (critical_path_ms, tail_stage) = critical_path(&stages);
let total_queued: u64 = stages.iter().map(|stage| stage.queue_ms).sum();

println!("critical path ms = {}", critical_path_ms);
println!("tail stage = {}", tail_stage);
println!("queued ms = {}", total_queued);
}
```

Exercises

Design saturation metrics, trace tail-latency incidents, identify retry storms, and capacity-plan distributed workers

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 37

CUDA and GPU Acceleration

When GPU acceleration pays, safe launch wrappers, device-memory ownership, transfer and launch overhead, profiling, and async or distributed integration

Overview

GPU acceleration is justified by throughput, transfer cost, memory layout, and launch overhead. This chapter covers Rust-side control of accelerator work without hiding those costs.

Examples

Example 37-1. `cuda_gpu_kernel_launch_wrapper.rs` — *Cuda gpu kernel launch wrapper*

```

#[derive(Debug, Clone, Copy)]
struct LaunchConfig {
    threads_per_block: u32,
    blocks: u32,
}

#[derive(Debug)]
struct DeviceBuffer {
    len: usize,
    bytes: usize,
}

impl DeviceBuffer {
    fn for_f32(len: usize) -> Self {
        Self {
            len,
            bytes: len * std::mem::size_of::<f32>(),
        }
    }
}

fn build_launch_config(len: usize, threads_per_block: u32) -> Result<LaunchConfig, &'static str> {
    if len == 0 {
        return Err("empty input");
    }

    if threads_per_block == 0 {
        return Err("threads_per_block must be > 0");
    }

    let blocks = ((len as u32) + threads_per_block - 1) / threads_per_block;

    Ok(LaunchConfig {
        threads_per_block,
        blocks,
    })
}

unsafe fn raw_launch_vec_add(
    config: LaunchConfig,
    a: &DeviceBuffer,
    b: &DeviceBuffer,
    out: &mut DeviceBuffer,
) -> Result<(), &'static str> {
    if a.len != b.len || a.len != out.len {
        return Err("shape mismatch");
    }

    if config.blocks == 0 || config.threads_per_block == 0 {
        return Err("invalid launch");
    }

    Ok(())
}

fn launch_vec_add(len: usize, threads_per_block: u32) -> Result<(LaunchConfig, usize), &'static str> {
    let config = build_launch_config(len, threads_per_block)?;
    let a = DeviceBuffer::for_f32(len);
    let b = DeviceBuffer::for_f32(len);
    let mut out = DeviceBuffer::for_f32(len);

    unsafe {
        // SAFETY:

```

```

    // - this wrapper creates all three device buffers with the same logical length.
    // - build_launch_config guarantees nonzero block and thread counts.
    // - the raw launch does not outlive these local buffers in this demo.
    raw_launch_vec_add(config, &a, &b, &mut out)?;
}

Ok((config, a.bytes + b.bytes + out.bytes))
}

fn main() {
    let (config, device_bytes) = launch_vec_add(4_096, 256).unwrap();

    println!("blocks = {}", config.blocks);
    println!("threads = {}", config.threads_per_block);
    println!("device bytes = {}", device_bytes);
}

```

Example 37-2. cuda_gpu_transfer_budget.rs — Cuda gpu transfer budget

```

#[derive(Debug, Clone, Copy)]
struct Workload {
    elements: usize,
    flops_per_element: u64,
    input_buffers: usize,
    output_buffers: usize,
}

fn transfer_bytes(work: Workload) -> usize {
    work.elements * std::mem::size_of::<f32>() * (work.input_buffers + work.output_buffers)
}

fn arithmetic_intensity(work: Workload) -> f64 {
    work.flops_per_element as f64
    / (std::mem::size_of::<f32>() as f64 * (work.input_buffers + work.output_buffers) as f64)
}

fn should_use_gpu(work: Workload, launch_us: u64) -> bool {
    let bytes = transfer_bytes(work);
    let intensity = arithmetic_intensity(work);

    bytes >= 8_000_000 && intensity >= 4.0 && launch_us <= 50
}

fn main() {
    let work = Workload {
        elements: 1_000_000,
        flops_per_element: 64,
        input_buffers: 2,
        output_buffers: 1,
    };
    let launch_us = 25_u64;

    println!("transfer bytes = {}", transfer_bytes(work));
    println!("intensity = {:.2}", arithmetic_intensity(work));
    println!("gpu faster = {}", should_use_gpu(work, launch_us));
}

```

Exercises

Estimate transfer cost, sketch a safe kernel wrapper, and choose CPU, Rayon, MPI, or CUDA from workload shape

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 38

Merkle Tree Games and Challenges

Hash trees, ownership-aware construction, inclusion proofs, persistent variants, parallel construction, serialization, and production challenges around content-addressed storage and distributed verification

Overview

Hash trees, ownership-aware construction, inclusion proofs, persistent variants, parallel construction, serialization, and production challenges around content-addressed storage and distributed verification

Examples

Example 38-1. merkle_tree_basic_proof.rs — Merkle tree basic proof

```

type Hash = u32;

#[derive(Debug, Clone, Copy)]
struct ProofStep {
    sibling: Hash,
    sibling_is_left: bool,
}

fn hash_bytes(tag: u8, bytes: &[u8]) -> Hash {
    let mut hash = 2_166_136_261_u32 ^ tag as u32;

    for &byte in bytes {
        hash ^= byte as u32;
        hash = hash.wrapping_mul(16_777_619);
    }

    hash
}

fn hash_leaf(text: &str) -> Hash {
    hash_bytes(0, text.as_bytes())
}

fn hash_node(left: Hash, right: Hash) -> Hash {
    let mut bytes = [0_u8; 8];
    bytes[..4].copy_from_slice(&left.to_le_bytes());
    bytes[4..].copy_from_slice(&right.to_le_bytes());
    hash_bytes(1, &bytes)
}

#[derive(Debug)]
struct MerkleTree {
    levels: Vec<Vec<Hash>>,
}

impl MerkleTree {
    fn from_leaves(leaves: &[&str]) -> Self {
        assert!(!leaves.is_empty(), "a Merkle tree needs at least one leaf");

        let mut levels = vec![leaves.iter().map(|leaf| hash_leaf(leaf)).collect::<Vec<_>>()];

        while levels.last().unwrap().len() > 1 {
            let current = levels.last().unwrap();
            let mut next = Vec::with_capacity((current.len() + 1) / 2);

            for pair in current.chunks(2) {
                let left = pair[0];
                let right = if pair.len() == 2 { pair[1] } else { pair[0] };
                next.push(hash_node(left, right));
            }

            levels.push(next);
        }

        Self { levels }
    }

    fn root(&self) -> Hash {
        self.levels.last().unwrap()[0]
    }

    fn proof(&self, mut index: usize) -> Vec<ProofStep> {
        let mut proof = Vec::new();

```

```

    for level in &self.levels[..self.levels.len() - 1] {
        let sibling_index = if index % 2 == 0 {
            (index + 1).min(level.len() - 1)
        } else {
            index - 1
        };

        proof.push(ProofStep {
            sibling: level[sibling_index],
            sibling_is_left: sibling_index < index,
        });

        index /= 2;
    }

    proof
}

fn verify_proof(leaf: &str, proof: &[ProofStep], expected_root: Hash) -> bool {
    let mut acc = hash_leaf(leaf);

    for step in proof {
        acc = if step.sibling_is_left {
            hash_node(step.sibling, acc)
        } else {
            hash_node(acc, step.sibling)
        };
    }

    acc == expected_root
}

fn main() {
    let leaves = ["alpha", "beta", "gamma", "delta"];
    let tree = MerkleTree::from_leaves(&leaves);
    let proof = tree.proof(2);

    println!("leaf count = {}", tree.levels[0].len());
    println!("proof len = {}", proof.len());
    println!("verified = {}", verify_proof("gamma", &proof, tree.root()));
}

```


Example 38-2. merkle_parallel_levels.rs — Merkle parallel levels

```

type Hash = u32;

fn hash_bytes(tag: u8, bytes: &[u8]) -> Hash {
    let mut hash = 2_166_136_261_u32 ^ tag as u32;

    for &byte in bytes {
        hash ^= byte as u32;
        hash = hash.wrapping_mul(16_777_619);
    }

    hash
}

fn hash_leaf(text: &str) -> Hash {
    hash_bytes(0, text.as_bytes())
}

fn hash_node(left: Hash, right: Hash) -> Hash {
    let mut bytes = [0_u8; 8];
    bytes[..4].copy_from_slice(&left.to_le_bytes());
    bytes[4..].copy_from_slice(&right.to_le_bytes());
    hash_bytes(1, &bytes)
}

fn next_level(level: &[Hash]) -> Vec<Hash> {
    let mut next = Vec::with_capacity((level.len() + 1) / 2);

    for pair in level.chunks(2) {
        let left = pair[0];
        let right = if pair.len() == 2 { pair[1] } else { pair[0] };
        next.push(hash_node(left, right));
    }

    next
}

fn next_level_parallel(level: &[Hash]) -> Vec<Hash> {
    if level.len() <= 2 {
        return next_level(level);
    }

    let pair_count = (level.len() + 1) / 2;
    let left_pairs = pair_count / 2;
    let split = (left_pairs * 2).max(2).min(level.len());

    std::thread::scope(|scope| {
        let left = &level[..split];
        let right = &level[split..];

        let left_handle = scope.spawn(move || next_level(left));
        let right_handle = scope.spawn(move || next_level(right));

        let mut out = left_handle.join().unwrap();
        out.extend(right_handle.join().unwrap());
        out
    })
}

fn build_root(leaves: &[&str], reducer: fn(&[Hash]) -> Vec<Hash>) -> (Hash, usize) {
    let mut level: Vec<Hash> = leaves.iter().map(|leaf| hash_leaf(leaf)).collect();
    let mut levels = 1;

    while level.len() > 1 {

```

```
        level = reducer(&level);
        levels += 1;
    }

    (level[0], levels)
}

fn main() {
    let leaves = ["a", "b", "c", "d", "e", "f", "g", "h"];

    let (serial_root, levels) = build_root(&leaves, next_level);
    let (parallel_root, _) = build_root(&leaves, next_level_parallel);

    println!("leaves = {}", leaves.len());
    println!("levels = {}", levels);
    println!("roots match = {}", serial_root == parallel_root);
}
```

Exercises

Implement a Merkle tree API, verify inclusion proofs, and design parallel, tamper-evident, and distributed verification systems

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 39

Graph Search Games

Graph representations in Rust, BFS and DFS, Dijkstra and A, ownership-friendly graph models, arena-backed graphs, parallel traversal, and challenge tracks for mazes, dependency resolution, and distributed search*

Overview

Graph representations in Rust, BFS and DFS, Dijkstra and A*, ownership-friendly graph models, arena-backed graphs, parallel traversal, and challenge tracks for mazes, dependency resolution, and distributed search

Examples

Example 39-1. graph_search_bfs_handles.rs — Graph search bfs handles

```

use std::collections::VecDeque;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct NodeId(usize);

#[derive(Debug)]
struct Node {
    name: &'static str,
    edges: Vec<NodeId>,
}

#[derive(Default, Debug)]
struct Graph {
    nodes: Vec<Node>,
}

impl Graph {
    fn add_node(&mut self, name: &'static str) -> NodeId {
        let id = NodeId(self.nodes.len());
        self.nodes.push(Node {
            name,
            edges: Vec::new(),
        });
        id
    }

    fn add_edge(&mut self, from: NodeId, to: NodeId) {
        self.nodes[from.0].edges.push(to);
    }

    fn name(&self, id: NodeId) -> &'static str {
        self.nodes[id.0].name
    }

    fn names(&self, ids: &[NodeId]) -> Vec<&'static str> {
        ids.iter().map(|&id| self.name(id)).collect()
    }

    fn bfs_order(&self, start: NodeId) -> Vec<NodeId> {
        let mut visited = vec![false; self.nodes.len()];
        let mut queue = VecDeque::new();
        let mut order = Vec::new();

        visited[start.0] = true;
        queue.push_back(start);

        while let Some(id) = queue.pop_front() {
            order.push(id);

            for &next in &self.nodes[id.0].edges {
                if !visited[next.0] {
                    visited[next.0] = true;
                    queue.push_back(next);
                }
            }
        }

        order
    }

    fn shortest_hops(&self, start: NodeId, goal: NodeId) -> Option<usize> {
        let mut dist = vec![usize::MAX; self.nodes.len()];
        let mut queue = VecDeque::new();

```

```

        dist[start.0] = 0;
        queue.push_back(start);

        while let Some(id) = queue.pop_front() {
            if id == goal {
                return Some(dist[id.0]);
            }

            for &next in &self.nodes[id.0].edges {
                if dist[next.0] == usize::MAX {
                    dist[next.0] = dist[id.0] + 1;
                    queue.push_back(next);
                }
            }
        }

        None
    }
}

fn main() {
    let mut graph = Graph::default();

    let api = graph.add_node("api");
    let auth = graph.add_node("auth");
    let billing = graph.add_node("billing");
    let search = graph.add_node("search");

    graph.add_edge(api, auth);
    graph.add_edge(api, billing);
    graph.add_edge(auth, search);
    graph.add_edge(billing, search);

    let order = graph.bfs_order(api);
    let hops = graph.shortest_hops(api, search).unwrap_or(0);

    println!("bfs = {}", graph.names(&order).join(", "));
    println!("path hops = {}", hops);
}

```

Example 39-2. graph_search_dijkstra_astar.rs — Graph search dijkstra astar

```

use std::cmp::Ordering;
use std::collections::BinaryHeap;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash, PartialOrd, Ord)]
struct NodeId(usize);

#[derive(Clone, Copy, Debug)]
struct Edge {
    to: NodeId,
    cost: u32,
}

#[derive(Debug)]
struct Node {
    name: &'static str,
    x: i32,
    y: i32,
    edges: Vec<Edge>,
}

#[derive(Default)]
struct WeightedGraph {
    nodes: Vec<Node>,
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct State {
    cost: u32,
    node: NodeId,
    estimate: u32,
}

impl Ord for State {
    fn cmp(&self, other: &Self) -> Ordering {
        other
            .estimate
            .cmp(&self.estimate)
            .then_with(|| other.cost.cmp(&self.cost))
            .then_with(|| other.node.cmp(&self.node))
    }
}

impl PartialOrd for State {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))
    }
}

impl WeightedGraph {
    fn add_node(&mut self, name: &'static str, x: i32, y: i32) -> NodeId {
        let id = NodeId(self.nodes.len());
        self.nodes.push(Node {
            name,
            x,
            y,
            edges: Vec::new(),
        });
        id
    }

    fn add_edge(&mut self, from: NodeId, to: NodeId, cost: u32) {
        self.nodes[from.0].edges.push(Edge { to, cost });
    }
}

```

```

fn heuristic(&self, left: NodeId, right: NodeId) -> u32 {
    self.nodes[left.0].x.abs_diff(self.nodes[right.0].x)
    + self.nodes[left.0].y.abs_diff(self.nodes[right.0].y)
}

fn dijkstra_cost(&self, start: NodeId, goal: NodeId) -> Option<u32> {
    let mut dist = vec![u32::MAX; self.nodes.len()];
    let mut heap = BinaryHeap::new();

    dist[start.0] = 0;
    heap.push(State {
        cost: 0,
        node: start,
        estimate: 0,
    });

    while let Some(State { cost, node, .. }) = heap.pop() {
        if node == goal {
            return Some(cost);
        }

        if cost > dist[node.0] {
            continue;
        }

        for edge in &self.nodes[node.0].edges {
            let next_cost = cost + edge.cost;
            if next_cost < dist[edge.to.0] {
                dist[edge.to.0] = next_cost;
                heap.push(State {
                    cost: next_cost,
                    node: edge.to,
                    estimate: next_cost,
                });
            }
        }
    }

    None
}

fn a_star_cost(&self, start: NodeId, goal: NodeId) -> Option<u32> {
    let mut best = vec![u32::MAX; self.nodes.len()];
    let mut heap = BinaryHeap::new();

    best[start.0] = 0;
    heap.push(State {
        cost: 0,
        node: start,
        estimate: self.heuristic(start, goal),
    });

    while let Some(State { cost, node, .. }) = heap.pop() {
        if node == goal {
            return Some(cost);
        }

        if cost > best[node.0] {
            continue;
        }

        for edge in &self.nodes[node.0].edges {
            let next_cost = cost + edge.cost;
            if next_cost < best[edge.to.0] {

```

```

        best[edge.to.0] = next_cost;
        heap.push(State {
            cost: next_cost,
            node: edge.to,
            estimate: next_cost + self.heuristic(edge.to, goal),
        });
    }
}

None
}

fn main() {
    let mut graph = WeightedGraph::default();

    let a = graph.add_node("A", 0, 0);
    let b = graph.add_node("B", 1, 0);
    let c = graph.add_node("C", 0, 1);
    let d = graph.add_node("D", 1, 1);
    let goal = graph.add_node("Goal", 2, 1);

    graph.add_edge(a, b, 2);
    graph.add_edge(a, c, 5);
    graph.add_edge(b, d, 2);
    graph.add_edge(c, d, 1);
    graph.add_edge(d, goal, 3);
    graph.add_edge(b, goal, 9);

    let dijkstra = graph.dijkstra_cost(a, goal).unwrap_or(0);
    let a_star = graph.a_star_cost(a, goal).unwrap_or(0);

    println!("dijkstra cost = {}", dijkstra);
    println!("a_star cost = {}", a_star);
}

```

Exercises

Implement BFS with stable indices, compare graph representations, and design maze, dependency, and distributed graph-search challenges

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 40

Matrix Optimization Games

Matrix storage layouts, cache-aware multiplication, SIMD opportunities, sparse vs dense tradeoffs, blocking and tiling, parallel matrix operations, GPU offload, and optimization challenge tracks

Overview

Matrix optimization depends on layout, tiling, sparsity, SIMD, thread budgets, and accelerator boundaries. This chapter turns those factors into measurable Rust implementation choices.

Examples

Example 40-1. `matrix_games_tiled_matmul.rs` — *Matrix games tiled matmul*

```

#[derive(Debug, Clone, PartialEq)]
struct Matrix {
    rows: usize,
    cols: usize,
    data: Vec<f32>,
}

impl Matrix {
    fn from_vec(rows: usize, cols: usize, data: Vec<f32>) -> Self {
        assert_eq!(data.len(), rows * cols);
        Self { rows, cols, data }
    }

    fn zeros(rows: usize, cols: usize) -> Self {
        Self {
            rows,
            cols,
            data: vec![0.0; rows * cols],
        }
    }

    fn offset(&self, row: usize, col: usize) -> usize {
        row * self.cols + col
    }

    fn get(&self, row: usize, col: usize) -> f32 {
        self.data[self.offset(row, col)]
    }

    fn set(&mut self, row: usize, col: usize, value: f32) {
        let index = self.offset(row, col);
        self.data[index] = value;
    }
}

fn matmul_naive(a: &Matrix, b: &Matrix) -> Matrix {
    assert_eq!(a.cols, b.rows);
    let mut out = Matrix::zeros(a.rows, b.cols);

    for i in 0..a.rows {
        for j in 0..b.cols {
            let mut acc = 0.0_f32;
            for k in 0..a.cols {
                acc += a.get(i, k) * b.get(k, j);
            }
            out.set(i, j, acc);
        }
    }

    out
}

fn matmul_tiled(a: &Matrix, b: &Matrix, tile: usize) -> Matrix {
    assert_eq!(a.cols, b.rows);
    let mut out = Matrix::zeros(a.rows, b.cols);
    let tile = tile.max(1);

    let mut ii = 0;
    while ii < a.rows {
        let mut kk = 0;
        while kk < a.cols {
            let mut jj = 0;
            while jj < b.cols {

```

```

        let i_end = (ii + tile).min(a.rows);
        let k_end = (kk + tile).min(a.cols);
        let j_end = (jj + tile).min(b.cols);

        for i in ii..i_end {
            for k in kk..k_end {
                let a_ik = a.get(i, k);
                for j in jj..j_end {
                    let current = out.get(i, j);
                    out.set(i, j, current + a_ik * b.get(k, j));
                }
            }
        }

        jj += tile;
    }
    kk += tile;
}
ii += tile;
}

out
}

fn approx_eq(left: &Matrix, right: &Matrix) -> bool {
    left.rows == right.rows
    && left.cols == right.cols
    && left
        .data
        .iter()
        .zip(&right.data)
        .all(|(l, r)| (l - r).abs() < 0.001)
}

fn checksum(matrix: &Matrix) -> f32 {
    matrix.data.iter().copied().sum()
}

fn main() {
    let a = Matrix::from_vec(
        3,
        3,
        vec![
            1.0, 2.0, 0.0,
            0.0, 1.0, 3.0,
            2.0, 0.0, 1.0,
        ],
    );

    let b = Matrix::from_vec(
        3,
        3,
        vec![
            3.0, 1.0, 2.0,
            2.0, 1.0, 0.0,
            1.0, 4.0, 2.0,
        ],
    );

    let naive = matmul_naive(&a, &b);
    let tiled = matmul_tiled(&a, &b, 2);

    println!("naive == tiled = {}", approx_eq(&naive, &tiled));
    println!("c[1,2] = {:.2}", tiled.get(1, 2));
}

```

```
println!("checksum = {:.2}", checksum(&tiled));  
}
```

Example 40-2. matrix_games_sparse_frontier.rs — *Matrix games sparse frontier*

```

#[derive(Debug)]
struct CsrMatrix {
    rows: usize,
    cols: usize,
    indptr: Vec<usize>,
    indices: Vec<usize>,
    data: Vec<f32>,
}

impl CsrMatrix {
    fn nnz(&self) -> usize {
        self.data.len()
    }

    fn dense_bytes(&self) -> usize {
        self.rows * self.cols * std::mem::size_of::<f32>()
    }
}

fn advance_frontier(csr: &CsrMatrix, frontier: &[f32]) -> Vec<u8> {
    assert_eq!(frontier.len(), csr.rows);

    let mut next = vec![0_u8; csr.cols];

    for row in 0..csr.rows {
        if frontier[row] == 0.0 {
            continue;
        }

        let start = csr.indptr[row];
        let end = csr.indptr[row + 1];

        for edge in start..end {
            let col = csr.indices[edge];
            if csr.data[edge] != 0.0 {
                next[col] = 1;
            }
        }
    }

    next
}

fn render(bits: &[u8]) -> String {
    bits.iter()
        .map(|bit| bit.to_string())
        .collect::<Vec<_>>()
        .join(",")
}

fn main() {
    let graph = CsrMatrix {
        rows: 5,
        cols: 5,
        indptr: vec![0, 2, 3, 5, 6, 6],
        indices: vec![1, 2, 3, 3, 4, 4],
        data: vec![1.0, 1.0, 1.0, 1.0, 1.0, 1.0],
    };

    let frontier = vec![1.0, 0.0, 1.0, 0.0, 0.0];
    let next = advance_frontier(&graph, &frontier);

    println!("nnz = {}", graph.nnz());
}

```

```
println!("next frontier = {}", render(&next));
println!("dense bytes = {}", graph.dense_bytes());
}
```

Exercises

Implement naive and tiled multiplication variants, choose sparse or dense representations, and design a fair CPU vs GPU benchmark tournament

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 41

Error Handling in Large Systems

Result and Option, typed error enums, thiserror and anyhow, context propagation, async and FFI boundaries, logging discipline, and error-contract design for production systems

Overview

Large systems need error contracts that separate domain failures, infrastructure failures, retry policy, and operator context. This chapter covers Rust error handling as a production interface.

Examples

Example 41-1. `error_handling_typed_contracts.rs` — *Error handling typed contracts*

```

use std::collections::HashMap;
use thiserror::Error;

#[derive(Debug, Clone)]
struct Order {
    id: &'static str,
    lines: Vec<u64>,
}

#[derive(Debug, Error)]
enum DomainError {
    #[error("order {order_id} has no lines")]
    EmptyOrder { order_id: &'static str },
}

#[derive(Debug, Error)]
enum InfraError {
    #[error("order {order_id} not found in store")]
    MissingOrder { order_id: &'static str },
}

#[derive(Debug, Error)]
enum ServiceError {
    #[error(transparent)]
    Domain(#[from] DomainError),
    #[error(transparent)]
    Infra(#[from] InfraError),
}

fn load_order<'a>(
    store: &'a HashMap<&'static str, Order>,
    order_id: &'static str,
) -> Result<&'a Order, InfraError> {
    store
        .get(order_id)
        .ok_or(InfraError::MissingOrder { order_id })
}

fn total_cents(order: &Order) -> Result<u64, DomainError> {
    if order.lines.is_empty() {
        return Err(DomainError::EmptyOrder {
            order_id: order.id,
        });
    }

    Ok(order.lines.iter().copied().sum())
}

fn bill_order(
    store: &HashMap<&'static str, Order>,
    order_id: &'static str,
) -> Result<u64, ServiceError> {
    let order = load_order(store, order_id)?;
    Ok(total_cents(order)?)
}

fn main() {
    let mut store = HashMap::new();
    store.insert(
        "ord-ok",
        Order {
            id: "ord-ok",
            lines: vec![1200_u64, 3000],
        }
    );
}

```



```

    },
);
store.insert(
    "ord-empty",
    Order {
        id: "ord-empty",
        lines: vec![],
    },
);

println!("ok = {}", bill_order(&store, "ord-ok").unwrap());
println!("missing = {}", bill_order(&store, "ord-missing").unwrap_err());
println!("empty = {}", bill_order(&store, "ord-empty").unwrap_err());
}

```

Example 41-2. `error_handling_async_context.rs` — *Error handling async context*

```

use anyhow::{Context, Result};
use std::io::{self, ErrorKind};

async fn read_artifact(name: &'static str) -> Result<&'static str> {
    match name {
        "manifest" => Ok("ready"),
        _ => Err(io::Error::new(ErrorKind::NotFound, "missing blob").into()),
    }
}

#[tokio::main]
async fn main() -> Result<> {
    let manifest = tokio::spawn(async {
        read_artifact("manifest")
            .await
            .context("loading manifest")
    })
    .await
    .context("manifest task failed")??;

    let failed = read_artifact("lockfile")
        .await
        .context("loading lockfile")
        .err()
        .map(|err| err.to_string())
        .unwrap_or_else(|| String::from("none"));

    println!("manifest = {}", manifest);
    println!("failed = {}", failed);

    Ok(())
}

```

Exercises

Convert panic-based logic into typed errors, separate domain and infrastructure failures, and add context to async propagation

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 42

Testing Advanced Rust Systems

Unit tests, integration tests, property testing, fuzzing, unsafe and async testing, distributed-system harnesses, golden files, snapshot testing, and benchmark regression budgets

Overview

Unit tests, integration tests, property testing, fuzzing, unsafe and async testing, distributed-system harnesses, golden files, snapshot testing, and benchmark regression budgets

Examples

Example 42-1. testing_property_invariant_harness.rs — *Testing property invariant harness*

```

#[derive(Debug, Clone, Copy)]
struct Quota {
    limit: u32,
    used: u32,
}

impl Quota {
    fn try_reserve(&mut self, qty: u32) -> bool {
        if self.used + qty > self.limit {
            return false;
        }

        self.used += qty;
        true
    }
}

fn invariant_holds(limit: u32, ops: &[u32]) -> bool {
    let mut quota = Quota { limit, used: 0 };

    for &qty in ops {
        let _accepted = quota.try_reserve(qty);
        if quota.used > quota.limit {
            return false;
        }
    }

    true
}

fn main() {
    let limit = 8_u32;
    let scenarios: &[&[u32]] = &[
        &[1_u32, 1, 1],
        &[4_u32, 4],
        &[5_u32, 4],
        &[2_u32, 2, 2, 2],
        &[8_u32],
    ];

    let all_valid = scenarios
        .iter()
        .all(|ops| invariant_holds(limit, ops));

    println!("cases = {}", scenarios.len());
    println!("all valid = {}", all_valid);
    println!("limit = {}", limit);
}

```

Example 42-2. `testing_async_idempotent_delivery_harness.rs` — *Testing async idempotent delivery harness*

```

use std::collections::HashSet;
use std::sync::Arc;

use tokio::sync::Mutex;

#[derive(Default)]
struct FakeStore {
    applied: Mutex<HashSet<'static str>>,
    duplicate_count: Mutex<usize>,
}

impl FakeStore {
    async fn apply_once(&self, event_id: &'static str) -> bool {
        let mut applied = self.applied.lock().await;
        if !applied.insert(event_id) {
            drop(applied);
            let mut duplicates = self.duplicate_count.lock().await;
            *duplicates += 1;
            return false;
        }

        true
    }

    async fn processed(&self) -> usize {
        self.applied.lock().await.len()
    }

    async fn duplicates(&self) -> usize {
        *self.duplicate_count.lock().await
    }
}

#[tokio::main]
async fn main() {
    let store = Arc::new(FakeStore::default());

    for event_id in ["evt-1", "evt-1", "evt-2"] {
        let _ = store.apply_once(event_id).await;
    }

    println!("processed = {}", store.processed().await);
    println!("duplicates = {}", store.duplicates().await);
    println!("deliveries = {}", 3);
}

```

Exercises

Choose the right test layer, design property and fuzz strategies, build async integration plans, and create a production-grade testing matrix

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 43

Observability

Logging, metrics, tracing, structured logs, OpenTelemetry, async and distributed tracing, profiling in production, alerting, SLOs, and observability-driven refactoring

Overview

Logging, metrics, tracing, structured logs, OpenTelemetry, async and distributed tracing, profiling in production, alerting, SLOs, and observability-driven refactoring

Examples

Example 43-1. observability_tracing_tokio_spans.rs — *Observability tracing tokio spans*

```

use tokio::sync::mpsc;
use tracing::{error, info, info_span, instrument, Instrument};

#[derive(Debug, Clone)]
struct Request {
    trace_id: &'static str,
    route: &'static str,
    bytes: usize,
}

#[instrument(
    name = "handle_request",
    skip(request),
    fields(trace_id = %request.trace_id, route = request.route, bytes = request.bytes)
)]
async fn handle_request(request: Request) -> Result<usize, &'static str> {
    if request.bytes == 0 {
        error!(status = "reject", "empty payload");
        return Err("empty payload");
    }

    info!(status = "ok", "request handled");
    Ok(request.bytes / 10)
}

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<Request>(4);

    tx.send(Request {
        trace_id: "req-7",
        route: "/score",
        bytes: 640,
    })
    .await
    .unwrap();

    tx.send(Request {
        trace_id: "req-8",
        route: "/score",
        bytes: 0,
    })
    .await
    .unwrap();

    tx.send(Request {
        trace_id: "req-9",
        route: "/health",
        bytes: 120,
    })
    .await
    .unwrap();

    drop(tx);

    let worker = info_span!("worker", worker = "ingest-a");
    let mut processed = 0_usize;
    let mut failures = 0_usize;
    let mut last_trace = "none";

    while let Some(request) = rx.recv().await {
        last_trace = request.trace_id;
    }

```

```

        match handle_request(request).instrument(worker.clone()).await {
            Ok(_units) => processed += 1,
            Err(_error) => failures += 1,
        }
    }

    println!("instrumented = {}", true);
    println!("processed = {}", processed);
    println!("failures = {}", failures);
    println!("last trace = {}", last_trace);
}

```

Example 43-2. observability_metrics_slo_window.rs — *Observability metrics slo window*

```

#[derive(Debug, Clone, Copy)]
struct WindowStats {
    requests: u64,
    errors: u64,
    queue_p95_ms: u64,
    handler_p95_ms: u64,
    busy_workers: u32,
    total_workers: u32,
}

fn success_rate(stats: &WindowStats) -> f64 {
    1.0 - stats.errors as f64 / stats.requests as f64
}

fn end_to_end_p95_ms(stats: &WindowStats) -> u64 {
    stats.queue_p95_ms + stats.handler_p95_ms
}

fn saturation(stats: &WindowStats) -> f64 {
    stats.busy_workers as f64 / stats.total_workers as f64
}

fn burn_alert(stats: &WindowStats) -> bool {
    end_to_end_p95_ms(stats) > 350 || success_rate(stats) < 0.995 || saturation(stats) > 0.90
}

fn main() {
    let stats = WindowStats {
        requests: 5_000,
        errors: 40,
        queue_p95_ms: 180,
        handler_p95_ms: 210,
        busy_workers: 19,
        total_workers: 20,
    };

    println!("e2e p95 = {}", end_to_end_p95_ms(&stats));
    println!("success rate = {:.4}", success_rate(&stats));
    println!("alert = {}", burn_alert(&stats));
}

```

Exercises

Instrument Tokio work with spans, design backpressure and latency metrics, and connect logs, metrics, and traces in incident narratives

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 44

Packaging and Deployment

Static binaries, cross-compilation, Docker images, minimal runtime containers, wasm and native-library packaging, CI/CD, supply-chain security, feature-gated builds, and release engineering

Overview

Static binaries, cross-compilation, Docker images, minimal runtime containers, wasm and native-library packaging, CI/CD, supply-chain security, feature-gated builds, and release engineering

Examples

Example 44-1. `packaging_target_matrix.rs` — *Packaging target matrix*

```
#[derive(Debug, Clone, Copy)]
enum BuildTarget {
    LinuxGnu,
    LinuxMusl,
    Wasm,
    NativeLib,
}

fn artifact_name(service: &str, target: BuildTarget) -> String {
    match target {
        BuildTarget::LinuxGnu => format!("{}", x86_64-unknown-linux-gnu", service),
        BuildTarget::LinuxMusl => format!("{}", x86_64-unknown-linux-musl", service),
        BuildTarget::Wasm => format!("{}", wasm32-unknown-unknown.wasm", service),
        BuildTarget::NativeLib => format!("lib{}.so", service),
    }
}

fn is_static(target: BuildTarget) -> bool {
    matches!(target, BuildTarget::LinuxMusl)
}

fn main() {
    let service = "payments";

    println!("musl = {}", artifact_name(service, BuildTarget::LinuxMusl));
    println!("wasm = {}", artifact_name(service, BuildTarget::Wasm));
    println!("static = {}", is_static(BuildTarget::LinuxMusl));
}
```

Example 44-2. `packaging_release_bundle.rs` — *Packaging release bundle*

```

#[derive(Debug)]
struct ReleaseBundle {
    profile: &'static str,
    artifacts: Vec<&'static str>,
    features: Vec<&'static str>,
    sbom: bool,
    signature: bool,
}

fn build_bundle(profile: &'static str, metrics: bool, admin: bool) -> ReleaseBundle {
    let mut artifacts = vec!["linux-musl", "container"];
    let mut features = vec!["core"];

    if metrics {
        features.push("metrics");
    }

    if admin {
        artifacts.push("native-lib");
        features.push("admin");
    }

    ReleaseBundle {
        profile,
        artifacts,
        features,
        sbom: true,
        signature: true,
    }
}

fn main() {
    let bundle = build_bundle("release", true, false);

    println!("profile = {}", bundle.profile);
    println!("artifacts = {}", bundle.artifacts.join(", "));
    println!("features = {}", bundle.features.join(", "));
    println!("signed = {}", bundle.sbom && bundle.signature);
}

```

Exercises

Build a packaging matrix, design a feature-gated release workflow, add supply-chain checks to CI, and write a production-ready release plan

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 45

Capstone: A Distributed Rust System

Compose an end-to-end distributed Rust system: typed task envelopes, brokered work, replay-safe completion, Merkle verification, specialized graph and matrix lanes, optional acceleration, and rollout discipline

Overview

Compose an end-to-end distributed Rust system: typed task envelopes, brokered work, replay-safe completion, Merkle verification, specialized graph and matrix lanes, optional acceleration, and rollout discipline

Examples

Example 45-1. capstone_task_envelope_routing.rs — Capstone task envelope routing

```
#[derive(Debug, Clone)]
enum Workload {
    GraphSearch { start: u32, goal: u32 },
    MatrixTile { rows: usize, cols: usize, tile: usize },
}

#[derive(Debug, Clone)]
struct TaskEnvelope {
    task_id: &'static str,
    tenant: &'static str,
    attempt: u32,
    verification_root: u32,
    workload: Workload,
}

fn routing_key(workload: &Workload) -> &'static str {
    match workload {
        Workload::GraphSearch { .. } => "tasks.graph",
        Workload::MatrixTile { .. } => "tasks.matrix",
    }
}

fn main() {
    let task = TaskEnvelope {
        task_id: "task-7",
        tenant: "acme",
        attempt: 1,
        verification_root: 4242,
        workload: Workload::GraphSearch { start: 4, goal: 19 },
    };

    println!("task = {}", task.task_id);
    println!("route = {}", routing_key(&task.workload));
    println!("root = {}", task.verification_root);
    println!("tenant = {}", task.tenant);
}
```

Example 45-2. capstone_worker_pool_workloads.rs — Capstone worker pool workloads

```

use std::collections::VecDeque;

#[derive(Debug, Clone)]
enum Workload {
    GraphSearch { frontier: Vec<u32> },
    MatrixTile { left: [f32; 4], right: [f32; 4] },
}

#[derive(Debug, Clone)]
struct Job {
    id: &'static str,
    workload: Workload,
}

fn graph_units(frontier: &[u32]) -> u32 {
    frontier.iter().copied().sum()
}

fn matrix_checksum(left: [f32; 4], right: [f32; 4]) -> f32 {
    left.iter().zip(right).map(|(l, r)| *l * r).sum()
}

fn main() {
    let mut queue = VecDeque::from([
        Job {
            id: "graph-1",
            workload: Workload::GraphSearch {
                frontier: vec![2_u32, 3, 5],
            },
        },
        Job {
            id: "matrix-1",
            workload: Workload::MatrixTile {
                left: [1.0_f32, 2.0, 3.0, 4.0],
                right: [2.0_f32, 3.0, 4.0, 5.0],
            },
        },
    ]);

    let mut completed = 0_u32;
    let mut graph_total = 0_u32;
    let mut matrix_total = 0.0_f32;

    while let Some(job) = queue.pop_front() {
        match job.workload {
            Workload::GraphSearch { frontier } => {
                graph_total += graph_units(&frontier);
            }
            Workload::MatrixTile { left, right } => {
                matrix_total += matrix_checksum(left, right);
            }
        }
        completed += 1;
    }

    println!("completed = {}", completed);
    println!("graph units = {}", graph_total);
    println!("matrix checksum = {:.1}", matrix_total);
}

```

Exercises

Review the capstone like a staff engineer: architecture, milestones, queue budgets, replay safety, profiling evidence, and rollout gates

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 46

FastAPI-Style Web Apps, Swagger, and OpenAPI Codegen

Design typed Rust web APIs with extractors, state, OpenAPI and Swagger workflows, generated clients, auth, idempotency, and graceful shutdown—without leaking HTTP into the domain

Overview

Web API delivery needs typed handlers, documented contracts, generated transport code, and domain services isolated from HTTP concerns. This chapter builds that production boundary in Rust.

Examples

Example 46-1. `fastapi_style_handler_service_boundary.rs` — *Fastapi style handler service boundary*

```

#[derive(Debug, Clone)]
struct CreateInvoiceRequest {
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]
struct CreateInvoiceCommand {
    tenant: String,
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]
struct Invoice {
    id: String,
    tenant: String,
    total_cents: u64,
}

#[derive(Debug)]
enum DomainError {
    EmptyInvoice,
    Unauthorized,
}

#[derive(Debug)]
enum ApiError {
    BadRequest(&'static str),
    Forbidden,
}

struct InvoiceService;

impl InvoiceService {
    fn create_invoice(&self, cmd: CreateInvoiceCommand) -> Result<Invoice, DomainError> {
        if cmd.line_totals.is_empty() {
            return Err(DomainError::EmptyInvoice);
        }

        let total_cents: u64 = cmd.line_totals.iter().copied().sum();

        Ok(Invoice {
            id: format!("inv-{}", cmd.customer_id),
            tenant: cmd.tenant,
            total_cents,
        })
    }
}

struct ApiState {
    invoices: InvoiceService,
    service_name: String,
}

#[derive(Debug)]
struct AuthenticatedUser {
    tenant: String,
    can_create: bool,
}

#[derive(Debug)]
struct CreatedInvoiceResponse {

```

```

    status: u16,
    request_id: String,
    invoice_id: String,
    tenant: String,
    total_cents: u64,
}

fn create_invoice_handler(
    state: &ApiState,
    actor: AuthenticatedUser,
    request_id: &str,
    body: CreateInvoiceRequest,
) -> Result<CreatedInvoiceResponse, ApiError> {
    if !actor.can_create {
        return Err(ApiError::Forbidden);
    }

    let cmd = CreateInvoiceCommand {
        tenant: actor.tenant,
        customer_id: body.customer_id,
        line_totals: body.line_totals,
    };

    let created = state
        .invoices
        .create_invoice(cmd)
        .map_err(|err| match err {
            DomainError::EmptyInvoice => ApiError::BadRequest("invoice must contain at least one line"),
            DomainError::Unauthorized => ApiError::Forbidden,
        })?;

    Ok(CreatedInvoiceResponse {
        status: 201,
        request_id: request_id.to_string(),
        invoice_id: created.id,
        tenant: created.tenant,
        total_cents: created.total_cents,
    })
}

fn main() {
    let state = ApiState {
        invoices: InvoiceService,
        service_name: String::from("billing-api"),
    };
    let actor = AuthenticatedUser {
        tenant: String::from("acme"),
        can_create: true,
    };
    let request = CreateInvoiceRequest {
        customer_id: String::from("42"),
        line_totals: vec![1_200_u64, 3_000],
    };

    let created = create_invoice_handler(&state, actor, "req-7", request).unwrap();

    println!("status = {}", created.status);
    println!("invoice = {}", created.invoice_id);
    println!("total cents = {}", created.total_cents);
    println!("request id = {}", created.request_id);
}

```


Example 46-2. fastapi_style_openapi_codegen_scaffold.rs — *Fastapi style openapi codegen scaffold*

```

#[derive(Debug, Clone)]
struct ApiSchema {
    name: &'static str,
    required_fields: &'static [&'static str],
}

#[derive(Debug, Clone)]
struct ApiOperation {
    method: &'static str,
    path: &'static str,
    operation_id: &'static str,
    request: Option<ApiSchema>,
    response: ApiSchema,
    auth: bool,
}

#[derive(Debug)]
struct OpenApiDoc {
    title: &'static str,
    version: &'static str,
    swagger_ui_path: &'static str,
    operations: Vec<ApiOperation>,
}

#[derive(Debug)]
struct CodegenPlan {
    client_methods: Vec<&'static str>,
    server_stubs: usize,
}

fn build_doc() -> OpenApiDoc {
    OpenApiDoc {
        title: "Billing API",
        version: "2026-04",
        swagger_ui_path: "/docs",
        operations: vec![
            ApiOperation {
                method: "POST",
                path: "/v1/invoices",
                operation_id: "create_invoice",
                request: Some(ApiSchema {
                    name: "CreateInvoiceRequest",
                    required_fields: &["customer_id", "line_totals"],
                }),
                response: ApiSchema {
                    name: "CreatedInvoiceResponse",
                    required_fields: &["invoice_id", "total_cents"],
                },
                auth: true,
            },
            ApiOperation {
                method: "GET",
                path: "/v1/invoices/{id}",
                operation_id: "get_invoice",
                request: None,
                response: ApiSchema {
                    name: "InvoiceResponse",
                    required_fields: &["invoice_id", "total_cents"],
                },
                auth: true,
            },
        ],
    }
}

```

```
}

fn generate_code(doc: &OpenApiDoc) -> CodegenPlan {
    let client_methods = doc
        .operations
        .iter()
        .map(|op| op.operation_id)
        .collect::<Vec<_>>();

    CodegenPlan {
        server_stubs: doc.operations.len(),
        client_methods,
    }
}

fn main() {
    let doc = build_doc();
    let plan = generate_code(&doc);

    println!("operations = {}", doc.operations.len());
    println!("swagger = {}", doc.swagger_ui_path);
    println!("client methods = {}", plan.client_methods.join(", "));
    println!("server stubs = {}", plan.server_stubs);
}
```

Exercises

Translate FastAPI-style ergonomics into Rust seams, choose an OpenAPI workflow, and prevent documentation or codegen drift

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 47

gRPC Services with Protobuf and Service API Codegen

Treat protobuf as the contract: unary and streaming gRPC shapes, generated code, transport-to-domain mapping, deadlines, retries, metadata, and polyglot compatibility

Overview

Treat protobuf as the contract: unary and streaming gRPC shapes, generated code, transport-to-domain mapping, deadlines, retries, metadata, and polyglot compatibility

Examples

Example 47-1. `grpc_transport_to_domain_mapping.rs` — *Grpc transport to domain mapping*

```

#[derive(Debug, Clone)]
struct CreateInvoiceRequest {
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug, Clone)]
struct CreateInvoiceCommand {
    tenant: String,
    customer_id: String,
    line_totals: Vec<u64>,
}

#[derive(Debug)]
enum TransportError {
    MissingCustomer,
    EmptyInvoice,
}

fn into_command(
    request: CreateInvoiceRequest,
    tenant: &str,
) -> Result<CreateInvoiceCommand, TransportError> {
    if request.customer_id.trim().is_empty() {
        return Err(TransportError::MissingCustomer);
    }

    if request.line_totals.is_empty() {
        return Err(TransportError::EmptyInvoice);
    }

    Ok(CreateInvoiceCommand {
        tenant: tenant.to_string(),
        customer_id: request.customer_id,
        line_totals: request.line_totals,
    })
}

struct InvoiceService;

impl InvoiceService {
    fn create(&self, command: &CreateInvoiceCommand) -> String {
        format!("inv-{}", command.customer_id)
    }
}

fn main() {
    let request = CreateInvoiceRequest {
        customer_id: String::from("cust-7"),
        line_totals: vec![1_200_u64, 3_000],
    };

    let command = into_command(request, "acme").unwrap();
    let total_cents: u64 = command.line_totals.iter().copied().sum();
    let service = InvoiceService;
    let invoice_id = service.create(&command);

    println!("tenant = {}", command.tenant);
    println!("customer = {}", command.customer_id);
    println!("total cents = {}", total_cents);
    println!("invoice = {}", invoice_id);
}

```

Example 47-2. `grpc_status_deadline_retry.rs` — *Grpc status deadline retry*

```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum GrpcCode {
    InvalidArgument,
    DeadlineExceeded,
    Unavailable,
    Cancelled,
}

#[derive(Debug, Clone, Copy)]
struct GrpcStatus {
    code: GrpcCode,
    message: &'static str,
}

#[derive(Debug, Clone, Copy)]
struct RetryDecision {
    retry: bool,
    reason: &'static str,
}

fn classify(status: GrpcStatus, idempotent: bool) -> RetryDecision {
    match status.code {
        GrpcCode::Unavailable | GrpcCode::DeadlineExceeded if idempotent => RetryDecision {
            retry: true,
            reason: "transient",
        },
        GrpcCode::Cancelled => RetryDecision {
            retry: false,
            reason: "caller_cancelled",
        },
        _ => RetryDecision {
            retry: false,
            reason: "do_not_retry",
        },
    }
}

fn should_abort(deadline_ms: u64, elapsed_ms: u64, cancelled: bool) -> bool {
    cancelled || elapsed_ms >= deadline_ms
}

fn main() {
    let unavailable = GrpcStatus {
        code: GrpcCode::Unavailable,
        message: "peer restarting",
    };
    let invalid = GrpcStatus {
        code: GrpcCode::InvalidArgument,
        message: "line_totals must not be empty",
    };

    let retry_unavailable = classify(unavailable, true);
    let retry_invalid = classify(invalid, true);

    println!("retry unavailable = {}", retry_unavailable.retry);
    println!("retry invalid = {}", retry_invalid.retry);
    println!("cancelled = {}", should_abort(150, 120, true));
}

```

Exercises

Choose the right RPC shape, contain generated types to the edge, and test schema evolution, deadlines, and streaming backpressure

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 48

WebSockets and Long-Lived Connections

Model long-lived connection systems with explicit read/write ownership, bounded fan-out, versioned envelopes, reconnect policy, and graceful shutdown

Overview

Model long-lived connection systems with explicit read/write ownership, bounded fan-out, versioned envelopes, reconnect policy, and graceful shutdown

Examples

Example 48-1. websocket_connection_io_split.rs — *Websocket connection io split*

```

use tokio::sync::{mpsc, watch};

#[derive(Debug)]
enum InboundFrame {
    Text(String),
    Ping(u64),
    Close,
}

#[derive(Debug)]
enum OutboundFrame {
    Text(String),
    Pong(u64),
    Close,
}

#[tokio::main]
async fn main() {
    let (inbound_tx, mut inbound_rx) = mpsc::channel::<InboundFrame>(4);
    let (outbound_tx, mut outbound_rx) = mpsc::channel::<OutboundFrame>(4);
    let (shutdown_tx, mut shutdown_rx) = watch::channel(false);

    let reader = tokio::spawn(async move {
        inbound_tx
            .send(InboundFrame::Text(String::from("ready")))
            .await
            .unwrap();
        inbound_tx.send(InboundFrame::Ping(7)).await.unwrap();
        inbound_tx.send(InboundFrame::Close).await.unwrap();
    });

    let app = tokio::spawn(async move {
        let mut inbound = 0_u32;

        while let Some(frame) = inbound_rx.recv().await {
            inbound += 1;

            match frame {
                InboundFrame::Text(text) => {
                    outbound_tx
                        .send(OutboundFrame::Text(format!("echo:{}", text)))
                        .await
                        .unwrap();
                }
                InboundFrame::Ping(nonce) => {
                    outbound_tx.send(OutboundFrame::Pong(nonce)).await.unwrap();
                }
                InboundFrame::Close => {
                    let _ = outbound_tx.send(OutboundFrame::Close).await;
                    shutdown_tx.send(true).unwrap();
                    break;
                }
            }
        }

        inbound
    });

    let writer = tokio::spawn(async move {
        let mut outbound = 0_u32;

        loop {
            tokio::select! {

```



```
        _ = shutdown_rx.changed() => {
            if *shutdown_rx.borrow() {
                break outbound;
            }
        }
        Some(frame) = outbound_rx.recv() => {
            match frame {
                OutboundFrame::Text(_) | OutboundFrame::Pong(_) | OutboundFrame::Close => {
                    outbound += 1;
                }
            }
        }
    }
}

});

reader.await.unwrap();
let inbound = app.await.unwrap();
let outbound = writer.await.unwrap();

println!("inbound = {}", inbound);
println!("outbound = {}", outbound);
println!("closed = {}", true);
}
```

Example 48-2. websocket_bounded_fanout_slow_consumers.rs — *Websocket bounded fanout slow consumers*

```

use std::collections::{HashMap, VecDeque};

#[derive(Debug, Default)]
struct Client {
    pending: VecDeque<String>,
}

#[derive(Debug)]
struct Hub {
    clients: HashMap<&'static str, Client>,
    max_pending: usize,
    delivered: usize,
    evicted: Vec<&'static str>,
}

impl Hub {
    fn new(max_pending: usize) -> Self {
        Self {
            clients: HashMap::new(),
            max_pending,
            delivered: 0,
            evicted: Vec::new(),
        }
    }

    fn add_client(&mut self, id: &'static str, pending: usize) {
        let mut queue = VecDeque::new();
        for _ in 0..pending {
            queue.push_back(String::from("old"));
        }

        self.clients.insert(id, Client { pending: queue });
    }

    fn broadcast(&mut self, payload: &str) {
        let ids: Vec<_> = self.clients.keys().copied().collect();

        for id in ids {
            let at_limit = self
                .clients
                .get(id)
                .map(|client| client.pending.len() >= self.max_pending)
                .unwrap_or(false);

            if at_limit {
                self.clients.remove(id);
                self.evicted.push(id);
                continue;
            }

            if let Some(client) = self.clients.get_mut(id) {
                client.pending.push_back(payload.to_string());
                self.delivered += 1;
            }
        }
    }

    fn main() {
        let mut hub = Hub::new(2);
        hub.add_client("alpha", 0);
        hub.add_client("beta", 2);
        hub.add_client("gamma", 1);
    }
}

```

```
hub.broadcast("update");

println!("active = {}", hub.clients.len());
println!("evicted = {}", hub.evicted.join(", "));
println!("delivered = {}", hub.delivered);
}
```

Exercises

Select the right live transport, bound slow consumers, design reconnect semantics, and test connection storms and drains

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 49

HTTPS, TLS, and Secure Service Boundaries

Engineer secure service edges: TLS termination, rustls vs platform TLS, mTLS, certificate rotation, hardened cookies/CORS/HSTS, and safe debugging

Overview

Engineer secure service edges: TLS termination, rustls vs platform TLS, mTLS, certificate rotation, hardened cookies/CORS/HSTS, and safe debugging

Examples

Example 49-1. `https_tls_topology_policy.rs` — *Https tls topology policy*

```

#[derive(Debug, Clone, Copy)]
enum TlsTermination {
    LoadBalancer,
    ReverseProxy,
    InProcess,
}

#[derive(Debug, Clone, Copy)]
enum PeerAuth {
    Off,
    Mtls,
}

#[derive(Debug)]
struct ServiceEdge {
    external_tls: TlsTermination,
    internal_peer_auth: PeerAuth,
    trusted_proxy_hops: u8,
    app_protocol: &'static str,
}

impl ServiceEdge {
    fn external_mode(&self) -> &'static str {
        match self.external_tls {
            TlsTermination::LoadBalancer => "load-balancer",
            TlsTermination::ReverseProxy => "reverse-proxy",
            TlsTermination::InProcess => "in-process",
        }
    }

    fn internal_mode(&self) -> &'static str {
        match self.internal_peer_auth {
            PeerAuth::Off => "tls-only",
            PeerAuth::Mtls => "mtls",
        }
    }

    fn forwarded_proto_policy(&self) -> &'static str {
        if self.trusted_proxy_hops > 0 {
            "trusted-proxy-only"
        } else {
            "ignore-forwarded-proto"
        }
    }

    fn alpn(&self) -> &'static str {
        self.app_protocol
    }
}

fn main() {
    let edge = ServiceEdge {
        external_tls: TlsTermination::LoadBalancer,
        internal_peer_auth: PeerAuth::Mtls,
        trusted_proxy_hops: 1,
        app_protocol: "h2",
    };

    println!("external = {}", edge.external_mode());
    println!("internal = {}", edge.internal_mode());
    println!("forwarded proto = {}", edge.forwarded_proto_policy());
}

```

```
println!("alpn = {}", edge.alpn());  
}
```

Example 49-2. `https_tls_security_defaults.rs` — *Https tls security defaults*

```

#[derive(Debug)]
struct SessionCookiePolicy {
    name: &'static str,
    secure: bool,
    http_only: bool,
    same_site: &'static str,
}

impl SessionCookiePolicy {
    fn is_hardened(&self) -> bool {
        self.secure && self.http_only && self.same_site != "None"
    }
}

#[derive(Debug)]
struct HttpSecurityPolicy {
    redirect_http: bool,
    hsts_max_age_secs: u64,
    allowed_origin: &'static str,
    renew_before_days: u16,
}

impl HttpSecurityPolicy {
    fn cors_mode(&self) -> &'static str {
        if self.allowed_origin == "*" {
            "wildcard"
        } else {
            "locked-down"
        }
    }

    fn hsts_enabled(&self) -> bool {
        self.redirect_http && self.hsts_max_age_secs > 0
    }

    fn rotate_now(&self, days_left: u16) -> bool {
        days_left <= self.renew_before_days
    }
}

fn main() {
    let cookie = SessionCookiePolicy {
        name: "__Host-session",
        secure: true,
        http_only: true,
        same_site: "Lax",
    };

    let policy = HttpSecurityPolicy {
        redirect_http: true,
        hsts_max_age_secs: 63_072_000,
        allowed_origin: "https://app.example.com",
        renew_before_days: 14,
    };

    println!("cookie = {}", cookie.name);
    println!("cookie secure = {}", cookie.is_hardened());
    println!("cors = {}", policy.cors_mode());
    println!("hsts = {}", policy.hsts_enabled());
    println!("rotate now = {}", policy.rotate_now(21));
}

```

Exercises

Choose TLS ownership, harden edge defaults, design cert rotation and mTLS, and debug trust failures without weakening verification

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 50

libp2p and Peer-to-Peer Rust Systems

Compose peer identity, discovery, request-response/pub-sub, relay policy, sync, and observability into a reviewable Rust P2P architecture

Overview

Compose peer identity, discovery, request-response/pub-sub, relay policy, sync, and observability into a reviewable Rust P2P architecture

Examples

Example 50-1. libp2p_swarm_state_machine.rs — *Libp2p swarm state machine*

```

use std::collections::{BTreeSet, HashMap, VecDeque};

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
struct PeerId(&'static str);

#[derive(Debug, Clone)]
enum LocalCommand {
    Dial(PeerId),
    SendRequest {
        peer: PeerId,
        request_id: u64,
        body: String,
    },
    Publish {
        topic: &'static str,
        body: String,
    },
}

#[derive(Debug, Clone)]
enum NetworkEvent {
    ConnectionEstablished(PeerId),
    Response {
        peer: PeerId,
        request_id: u64,
        body: String,
    },
    Gossip {
        peer: PeerId,
        topic: &'static str,
        body: String,
    },
}

#[derive(Debug, Default)]
struct SwarmState {
    connected: BTreeSet<PeerId>,
    pending_requests: HashMap<u64, PeerId>,
    log: VecDeque<String>,
}

impl SwarmState {
    fn on_command(&mut self, command: LocalCommand) {
        match command {
            LocalCommand::Dial(peer) => {
                self.log.push_back(format!("dial {}", peer.0));
            }
            LocalCommand::SendRequest {
                peer,
                request_id,
                body,
            } => {
                self.pending_requests.insert(request_id, peer);
                self.log.push_back(format!("request {} {}", peer.0, body));
            }
            LocalCommand::Publish { topic, body } => {
                self.log.push_back(format!("publish {} {}", topic, body));
            }
        }
    }

    fn on_event(&mut self, event: NetworkEvent) {
        match event {

```

```

        NetworkEvent::ConnectionEstablished(peer) => {
            self.connected.insert(peer);
            self.log.push_back(format!("connected {}", peer.0));
        }
        NetworkEvent::Response {
            peer,
            request_id,
            body,
        } => {
            self.pending_requests.remove(&request_id);
            self.log.push_back(format!("response {} {}", peer.0, body));
        }
        NetworkEvent::Gossip { peer, topic, body } => {
            self.log
                .push_back(format!("gossip {} {} {}", topic, peer.0, body));
        }
    }
}

fn main() {
    let mut swarm = SwarmState::default();
    let peer = PeerId("peer-b");

    swarm.on_command(LocalCommand::Dial(peer));
    swarm.on_event(NetworkEvent::ConnectionEstablished(peer));
    swarm.on_command(LocalCommand::SendRequest {
        peer,
        request_id: 7,
        body: String::from("want-state"),
    });
    swarm.on_event(NetworkEvent::Response {
        peer,
        request_id: 7,
        body: String::from("state-v3"),
    });
    swarm.on_event(NetworkEvent::Gossip {
        peer,
        topic: "heads",
        body: String::from("tip=9"),
    });

    println!("connected = {}", swarm.connected.len());
    println!("pending = {}", swarm.pending_requests.len());
    println!(
        "last = {}",
        swarm.log.back().map(String::as_str).unwrap_or("none")
    );
}

```

Example 50-2. libp2p_state_sync_conflicts.rs — *Libp2p state sync conflicts*

```

#[derive(Debug, Clone, PartialEq, Eq)]
struct VersionedValue {
    version: u64,
    author: &'static str,
    value: String,
}

fn pick_newer(local: VersionedValue, remote: VersionedValue) -> VersionedValue {
    if remote.version > local.version {
        remote
    } else if remote.version < local.version {
        local
    } else if remote.author > local.author {
        remote
    } else {
        local
    }
}

fn merge_peer_updates(
    current: VersionedValue,
    updates: Vec<VersionedValue>,
) -> VersionedValue {
    updates.into_iter().fold(current, pick_newer)
}

fn main() {
    let current = VersionedValue {
        version: 3,
        author: "peer-a",
        value: String::from("allow-read"),
    };

    let merged = merge_peer_updates(
        current,
        vec![
            VersionedValue {
                version: 4,
                author: "peer-b",
                value: String::from("allow-write"),
            },
            VersionedValue {
                version: 4,
                author: "peer-c",
                value: String::from("allow-write+audit"),
            },
        ],
    );

    println!("version = {}", merged.version);
    println!("author = {}", merged.author);
    println!("value = {}", merged.value);
}

```

Exercises

Model swarm loops, bound discovery, plan NAT/relay fallback, and decide when P2P beats brokers or service APIs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 51

Zero-Knowledge Proofs for Rust Engineers

Separate statements, witnesses, constraints, transcripts, keys, proofs, and service boundaries so ZK systems stay reviewable in Rust

Overview

Separate statements, witnesses, constraints, transcripts, keys, proofs, and service boundaries so ZK systems stay reviewable in Rust

Examples

Example 51-1. `zkp_statement_witness_proof.rs` — *Zkp statement witness proof*

```

#[derive(Debug, Clone, Copy)]
struct Statement {
    public_total: u64,
    public_limit: u64,
}

#[derive(Debug, Clone, Copy)]
struct Witness {
    left: u64,
    right: u64,
}

#[derive(Debug, Clone, Copy)]
struct ProofArtifact {
    public_total: u64,
    public_limit: u64,
    proof_bytes_len: usize,
}

fn prove(statement: Statement, witness: Witness) -> Result<ProofArtifact, &'static str> {
    if witness.left + witness.right != statement.public_total {
        return Err("sum constraint failed");
    }

    if statement.public_total > statement.public_limit {
        return Err("public statement not admissible");
    }

    Ok(ProofArtifact {
        public_total: statement.public_total,
        public_limit: statement.public_limit,
        proof_bytes_len: 96,
    })
}

fn verify(statement: Statement, proof: ProofArtifact) -> bool {
    statement.public_total == proof.public_total
    && statement.public_limit == proof.public_limit
    && proof.proof_bytes_len > 0
}

fn main() {
    let statement = Statement {
        public_total: 45,
        public_limit: 50,
    };
    let witness = Witness { left: 20, right: 25 };
    let proof = prove(statement, witness).unwrap();

    println!("public total = {}", statement.public_total);
    println!("proof bytes = {}", proof.proof_bytes_len);
    println!("verified = {}", verify(statement, proof));
}

```

Example 51-2. `zkp_transcript_domain_separation.rs` — *Zkp transcript domain separation*

```

#[derive(Debug)]
struct Transcript {
    domain: &'static str,
    bytes: Vec<u8>,
}

impl Transcript {
    fn new(domain: &'static str) -> Self {
        Self {
            domain,
            bytes: domain.as_bytes().to_vec(),
        }
    }

    fn append_u64(&mut self, label: &str, value: u64) {
        self.bytes.extend_from_slice(label.as_bytes());
        self.bytes.extend_from_slice(&value.to_le_bytes());
    }

    fn challenge(&self) -> u64 {
        self.bytes.iter().fold(1_469_598_103_934_665_603_u64, |acc, byte| {
            acc.wrapping_mul(1_099_511_628_211).wrapping_add(*byte as u64)
        })
    }
}

fn challenge_for(domain: &'static str, public_total: u64, public_commitment: u64) -> u64 {
    let mut transcript = Transcript::new(domain);
    transcript.append_u64("public_total", public_total);
    transcript.append_u64("public_commitment", public_commitment);
    transcript.challenge()
}

fn main() {
    let domain = "billing-proof:v1";
    let prover_challenge = challenge_for(domain, 45, 9_001);
    let verifier_challenge = challenge_for(domain, 45, 9_001);
    let other_domain_challenge = challenge_for("inventory-proof:v1", 45, 9_001);

    println!("domain = {}", domain);
    println!("challenge match = {}", prover_challenge == verifier_challenge);
    println!("domain separation = {}", prover_challenge != other_domain_challenge);
}

```

Exercises

Keep public and private material distinct, isolate proving from verification, and threat-model proof-backed APIs

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 52

ZoKrates Workflows and Ethereum Verifiers

Run ZoKrates like a toolchain: compile/setup/witness/proof/export/verify stages, artifact custody, Ethereum verifier integration, and upgrade discipline

Overview

Proof workflows need controlled source programs, setup material, witness custody, generated verifiers, and deployment tracking. This chapter covers ZoKrates and Ethereum verification as staged production artifacts.

Examples

Example 52-1. `zokrates_workflow_command_plan.rs` — *Zokrates workflow command plan*

```

use std::path::PathBuf;

#[derive(Debug, Clone)]
struct Invocation {
    stage: &'static str,
    program: &'static str,
    args: Vec<String>,
    artifact: PathBuf,
}

fn plan_for(circuit: &str, witness_args: &[&str]) -> Vec<Invocation> {
    vec![
        Invocation {
            stage: "compile",
            program: "zokrates",
            args: vec![
                "compile".into(),
                "-i".into(),
                format!("{}", circuit),
                "-o".into(),
                circuit.into(),
            ],
            artifact: PathBuf::from(format!("artifacts/{}", circuit)),
        },
        Invocation {
            stage: "setup",
            program: "zokrates",
            args: vec!["setup".into(), "-i".into(), circuit.into()],
            artifact: PathBuf::from("artifacts/proving.key"),
        },
        Invocation {
            stage: "compute-witness",
            program: "zokrates",
            args: {
                let mut args = vec![
                    "compute-witness".into(),
                    "-i".into(),
                    circuit.into(),
                    "-a".into(),
                ];
                args.extend(witness_args.iter().map(|value| value.to_string()));
                args
            },
            artifact: PathBuf::from("artifacts/witness"),
        },
        Invocation {
            stage: "generate-proof",
            program: "zokrates",
            args: vec!["generate-proof".into(), "-i".into(), circuit.into()],
            artifact: PathBuf::from("artifacts/proof.json"),
        },
        Invocation {
            stage: "export-verifier",
            program: "zokrates",
            args: vec!["export-verifier".into(), "-i".into(), circuit.into()],
            artifact: PathBuf::from("artifacts/AgeCheckVerifier.sol"),
        },
        Invocation {
            stage: "verify",
            program: "zokrates",
            args: vec!["verify".into(), "-i".into(), circuit.into()],
            artifact: PathBuf::from("artifacts/verify.log"),
        },
    ],
}

```

```

    ]
}

fn render(invocation: &Invocation) -> String {
    format!("{}", invocation.program, invocation.args.join(" "))
}

fn main() {
    let plan = plan_for("age_check", &["18", "21"]);

    println!("steps = {}", plan.len());
    println!("compile = {}", render(&plan[0]));
    println!("proof = {}", render(&plan[3]));
    println!("contract = {}", plan[4].artifact.display());
}

```

Example 52-2. `zokrates_ethereum_verifier_boundary.rs` — *Zokrates ethereum verifier boundary*

```

#[derive(Debug, Clone)]
struct ProofBundle {
    circuit_id: String,
    verifier_contract: String,
    public_inputs: Vec<String>,
    proof_json_path: String,
    witness_path: String,
}

#[derive(Debug, Clone)]
struct VerifierCall {
    contract: String,
    function: &'static str,
    public_inputs_len: usize,
    proof_source: String,
}

fn to_verifier_call(bundle: &ProofBundle) -> VerifierCall {
    VerifierCall {
        contract: bundle.verifier_contract.clone(),
        function: "verifyTx",
        public_inputs_len: bundle.public_inputs.len(),
        proof_source: bundle.proof_json_path.clone(),
    }
}

fn main() {
    let bundle = ProofBundle {
        circuit_id: String::from("age-check:v2"),
        verifier_contract: String::from("contracts/AgeCheckVerifier.sol"),
        public_inputs: vec![String::from("45"), String::from("50")],
        proof_json_path: String::from("artifacts/proof.json"),
        witness_path: String::from("artifacts/witness"),
    };

    let call = to_verifier_call(&bundle);
    let witness_leaked = call.proof_source.contains("witness");

    println!("contract = {}", call.contract);
    println!("verifier fn = {}", call.function);
    println!("public inputs = {}", call.public_inputs_len);
    println!("witness leaked = {}", witness_leaked);
}

```

Exercises

Split build/release/runtime responsibilities, version artifacts, keep verifier requests witness-free, and plan audits and rollouts

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 53

EZKL, Verifiable LLM Inference, and GPU-Aware ZKML

Productize verifiable inference with explicit model artifacts, tokenization and quantization policy, bounded proving lanes, GPU-aware profiling, and verifier-safe APIs

Overview

Productize verifiable inference with explicit model artifacts, tokenization and quantization policy, bounded proving lanes, GPU-aware profiling, and verifier-safe APIs

Examples

Example 53-1. zkml_verification_boundary_types.rs — Zkml verification boundary types

```

#[derive(Debug, Clone)]
struct PublicClaim {
    model_id: String,
    quantization_bits: u8,
    token_count: usize,
    output_commitment: String,
}

#[derive(Debug, Clone)]
struct WitnessMaterial {
    prompt_tokens: Vec<u32>,
    attention_mask: Vec<u8>,
}

#[derive(Debug, Clone)]
struct ProofArtifact {
    circuit_id: String,
    proof_bytes_len: usize,
    verifier_key_id: String,
}

#[derive(Debug, Clone)]
struct VerificationRequest {
    claim: PublicClaim,
    proof: ProofArtifact,
}

fn build_verification_request(claim: PublicClaim, proof: ProofArtifact) -> VerificationRequest {
    VerificationRequest { claim, proof }
}

fn main() {
    let claim = PublicClaim {
        model_id: String::from("llm-int8:v3"),
        quantization_bits: 8,
        token_count: 128,
        output_commitment: String::from("out:9af1"),
    };

    let _witness = WitnessMaterial {
        prompt_tokens: vec![101_u32, 202, 303],
        attention_mask: vec![1_u8, 1, 1],
    };

    let proof = ProofArtifact {
        circuit_id: String::from("ezkl-circuit:v3"),
        proof_bytes_len: 2_048,
        verifier_key_id: String::from("vk:2026-04"),
    };

    let request = build_verification_request(claim, proof);

    println!("model = {}", request.claim.model_id);
    println!("public tokens = {}", request.claim.token_count);
    println!("proof bytes = {}", request.proof.proof_bytes_len);
    println!("witness kept private = {}", true);
}

```

Example 53-2. zkml_proving_queue_profile.rs — Zkml proving queue profile

```

#[derive(Debug, Clone)]
struct ZkmlJob {
    artifact_id: String,
    gpu_requested: bool,
    batch_size: usize,
}

#[derive(Debug, Clone, Copy)]
struct StageTimes {
    witness_ms: u64,
    prove_ms: u64,
    verify_ms: u64,
    transfer_ms: u64,
}

fn queue_name(job: &ZkmlJob) -> &'static str {
    if job.gpu_requested {
        "zkml.gpu"
    } else {
        "zkml.cpu"
    }
}

fn dominant_stage(times: &StageTimes) -> &'static str {
    [
        ("witness", times.witness_ms),
        ("prove", times.prove_ms),
        ("verify", times.verify_ms),
        ("transfer", times.transfer_ms),
    ]
    .into_iter()
    .max_by_key(|(_, ms)| *ms)
    .map(|(name, _)| name)
    .unwrap()
}

fn end_to_end_ms(times: &StageTimes) -> u64 {
    times.witness_ms + times.prove_ms + times.verify_ms + times.transfer_ms
}

fn main() {
    let job = ZkmlJob {
        artifact_id: String::from("model-sha256:abc123"),
        gpu_requested: true,
        batch_size: 4,
    };

    let times = StageTimes {
        witness_ms: 320,
        prove_ms: 1_280,
        verify_ms: 40,
        transfer_ms: 240,
    };

    println!("route = {}", queue_name(&job));
    println!("dominant = {}", dominant_stage(&times));
    println!("e2e ms = {}", end_to_end_ms(&times));
    println!("artifact = {}", job.artifact_id);
}

```

Exercises

Classify ZKML artifacts, separate proving from verification, profile bottlenecks honestly, and plan versioned rollout

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.

CHAPTER 54

no-std Rust and Constrained Runtime Derivatives

Design portable Rust surfaces across core, alloc, and std: fixed-capacity memory, DMA/MMIO ownership, embedded layering, and host-testable no_std crates

Overview

Constrained targets need explicit runtime surfaces for allocation, panic handling, IO, startup, and host calls. This chapter covers no_std Rust as a portability and assurance strategy.

Examples

Example 54-1. no_std_portable_surface.rs — No std portable surface

```
#[cfg(feature = "alloc")]
extern crate alloc;

#[cfg(feature = "alloc")]
use alloc::vec::Vec;

fn checksum(bytes: &[u8]) -> u32 {
    bytes.iter().map(|&byte| byte as u32).sum()
}

#[cfg(feature = "alloc")]
fn encode_frame(tag: u8, payload: &[u8]) -> Vec<u8> {
    let mut out = Vec::with_capacity(payload.len() + 1);
    out.push(tag);
    out.extend_from_slice(payload);
    out
}

#[cfg(feature = "std")]
fn write_diagnostic(service: &str, checksum: u32) -> String {
    format!("{:}:{:}", service, checksum)
}

fn main() {
    let payload = [1_u8, 2, 3];
    let sum = checksum(&payload);

    println!("portable modes = core|alloc|std");
    println!("checksum = {}", sum);
    println!("alloc-gated api = encode_frame");
}
```

Example 54-2. no_std_fixed_capacity_dma.rs — No std fixed capacity dma

```

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct SlotId(usize);

struct Pool<const SLOTS: usize, const BYTES: usize> {
    used: [bool; SLOTS],
    lens: [usize; SLOTS],
    data: [[u8; BYTES]; SLOTS],
}

impl<const SLOTS: usize, const BYTES: usize> Pool<SLOTS, BYTES> {
    const fn new() -> Self {
        Self {
            used: [false; SLOTS],
            lens: [0; SLOTS],
            data: [[0; BYTES]; SLOTS],
        }
    }

    fn alloc_copy(&mut self, bytes: &[u8]) -> Option<SlotId> {
        if bytes.len() > BYTES {
            return None;
        }

        let mut index = 0;
        while index < SLOTS {
            if !self.used[index] {
                self.used[index] = true;
                self.lens[index] = bytes.len();
                self.data[index][..bytes.len()].copy_from_slice(bytes);
                return Some(SlotId(index));
            }
            index += 1;
        }

        None
    }

    fn as_slice(&self, id: SlotId) -> &[u8] {
        &self.data[id.0][..self.lens[id.0]]
    }

    fn release(&mut self, id: SlotId) {
        self.used[id.0] = false;
        self.lens[id.0] = 0;
    }

    fn in_use(&self) -> usize {
        self.used.iter().filter(|&used| *used).count()
    }
}

fn main() {
    let mut pool = Pool::<2, 8>::new();

    let first = pool.alloc_copy(b"abc").unwrap();
    let _second = pool.alloc_copy(b"rust").unwrap();
    let overflow = pool.alloc_copy(b"more").is_none();
    let sent_bytes = pool.as_slice(first).len();

    pool.release(first);

    println!("in_use = {}", pool.in_use());
    println!("overflow = {}", overflow);
}

```

```
println!("sent bytes = {}", sent_bytes);  
}
```

Exercises

Choose the right runtime surface, design fixed-capacity paths, audit constrained unsafe code, and keep portable logic testable from the host

Suggested practice

- State the boundary or invariant first, then choose the Rust feature or refactor that matches it.
- Keep the answer concrete enough that another engineer could review the same design quickly.
- Prefer explicit ownership, pacing, and versioning rules over convenience hidden in one framework surface.